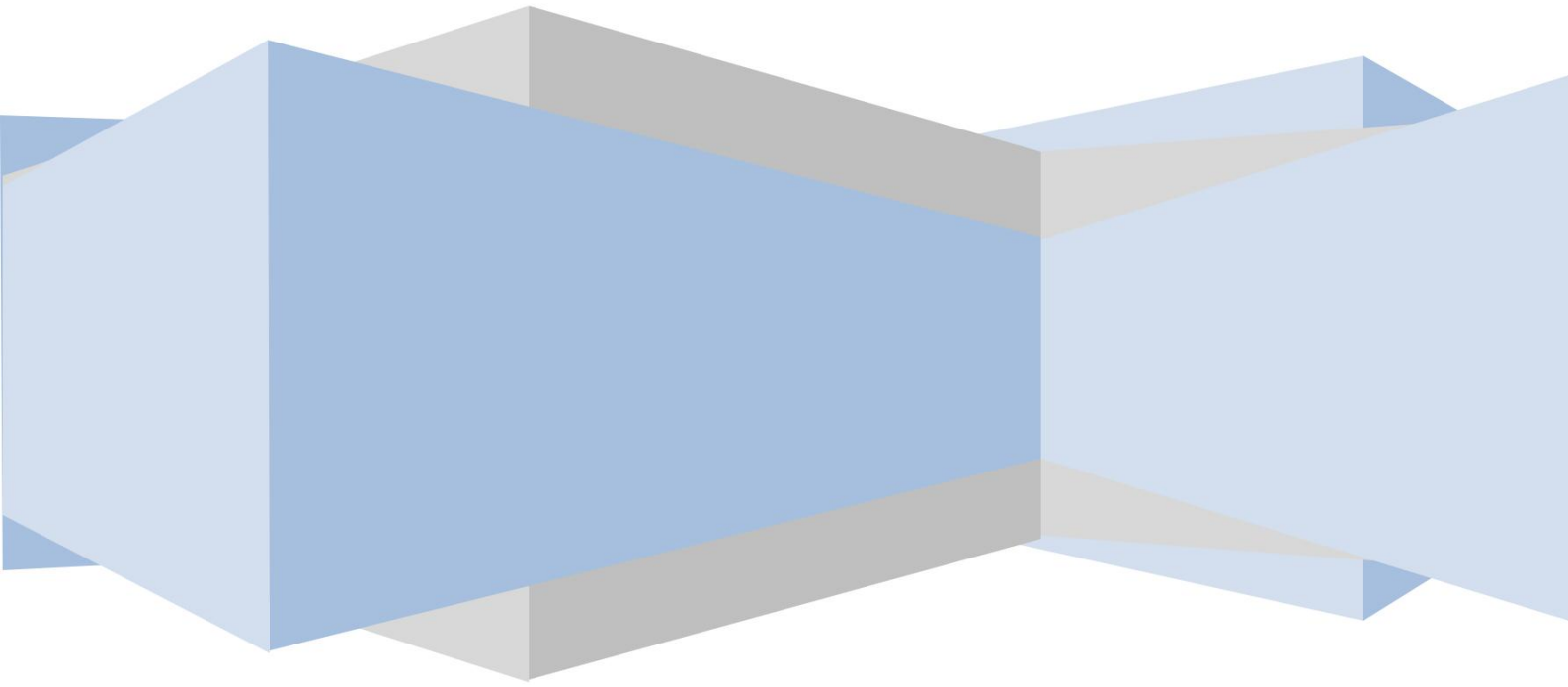
PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

UNIDAD 12: DESARROLLO DE JUEGOS CON UNITY



CURSO: 2º DAM

PROFESORA: SORAYA BLÁZQUEZ MORENO

ÍNDICE

Contenido

1.- Introducción 3
2.- Creación del proyecto y configuración inicial 3
3.- Importando Sprites 4
4.- Prefabs 7
5.- Personaje 8
6.- Animaciones 14
7.- Componente Animator 22
8.- Programación Saltar 27
9.- Programación Correr 33
10.- Bloques 36
11.- Notificaciones 39
12.- Detección de colisiones 41
13.- Ganando puntos 43
14.- Marcador de puntos 47
15.- Fondos/Scroll Parallax 48
16.- Fin del juego 52
17.- Pantalla de portada 59
18.- Música y sonidos 60
19.- Exportar el proyecto 63

1.- Introducción

Una vez que hemos entendido la teoría sobre los motores de juegos, se va a desarrollar un juego con el software Unity y el lenguaje de programación C#.

Lo primero que hay que hacer es instalar Unity en el siguiente enlace: https://unity.com/download .

Además, en el siguiente enlace hay un manual de Unity para las dudas que puedan surgir: https://docs.unity3d.com/es/530/Manual/UnityManual.html .

Una vez instalado el Unity vamos a conocer su interfaz accediendo al siguiente enlace https://docs.unity3d.com/es/530/Manual/LearningtheInterface.html .

Vamos a aprender cómo trabaja Unity y que posibilidades ofrece, desarrollando un juego 2D simple, pero muy completo.

Todo el proceso se describe en el siguiente videotutorial, que consta de 27 vídeos y es la referencia ampliada de los apartados de ese documento, que podéis consultar si en algún momento no tenéis suficiente con la explicación de estos apuntes: Creando un juego "Infinite Runner" en 2D con Unity 4.3.

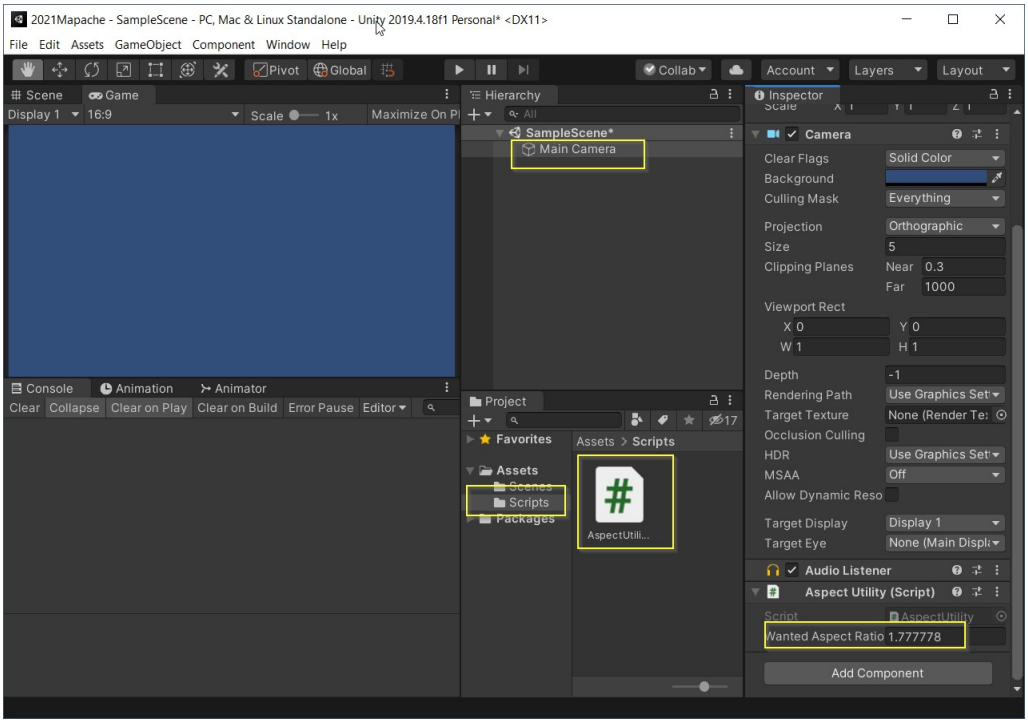
En el desarrollo del juego de plataforma aprenderemos a cargar y utilizar Sprites, crearemos al protagonista del juego uniendo todos sus elementos de forma jerárquica para posteriormente animarlo. Luego programaremos para que nuestro personaje salte y corra. También veremos cómo se generan los niveles de forma aleatoria y añadiremos ítems que el personaje irá cogiendo, además de hacer el marcador de puntos. Por último, añadiremos fondos al juego y cargaremos la puntuación máxima obtenida y finalizaremos añadiendo música y sonido

2.- Creación del proyecto y configuración inicial

1. **Crea un nuevo proyecto:** para ello en la opción File>New Proyect y selecciona un directorio vacío donde se guardará el juego.
2. **Configuración de la interfaz:** Elige el layout y movemos la pestaña de “Game” hacia abajo para tener las dos pantallas a la vez (Scene y Game) y por último añade la consola accediendo a Windows>General>Console. Para guardar esta

configuración de interfaz damos click en el tipo Layout Tall y seleccionamos “Save Layout“ y le ponemos un nombre.

3. **Tener una relación de aspecto independiente de la resolución del dispositivo:** Dentro de Assets, crea la carpeta Scripts e introduce en ella el siguiente script Script (**AspectUtility.cs)** y asígnaselo a la cámara (arrastrando). Este script permite tener una relación de aspecto independiente de la resolución del dispositivo en el que se ejecute el juego. Introduce en el parámetro del script Wanted Aspect Ratio el valor 1,777778 (este valor se calcula dividiendo una resolución entre otra, por ejemplo, si elijo 16:9, dividimos 16 entre 9 y este será el valor resultante).



A continuación:

- Guardamos la escena principal del juego. Para ello creamos en Asset otra carpeta llamada “Scenes” y en la opción File>Save as ponemos el nombre que queramos. También guardamos el proyecto en la opción File>Save Project.

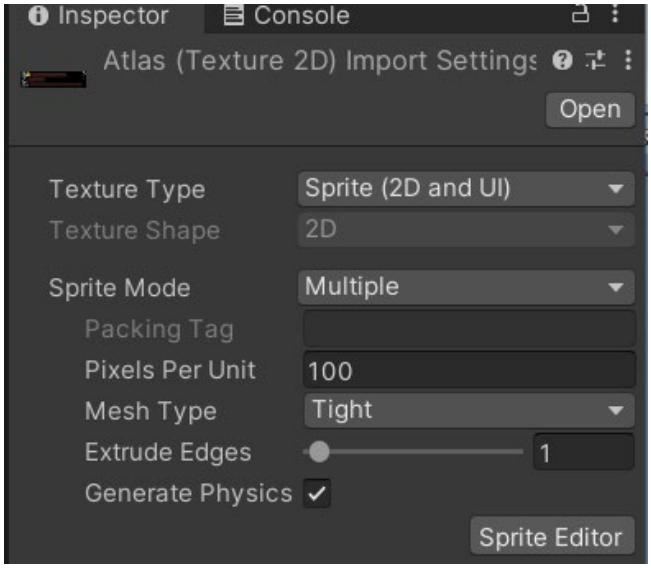
3.- Importando Sprites

Una **SpriteSheet** no es más que una imagen que contiene imágenes más pequeñas. A partir de ella, se pueden obtener cada imagen por separado de forma independiente,

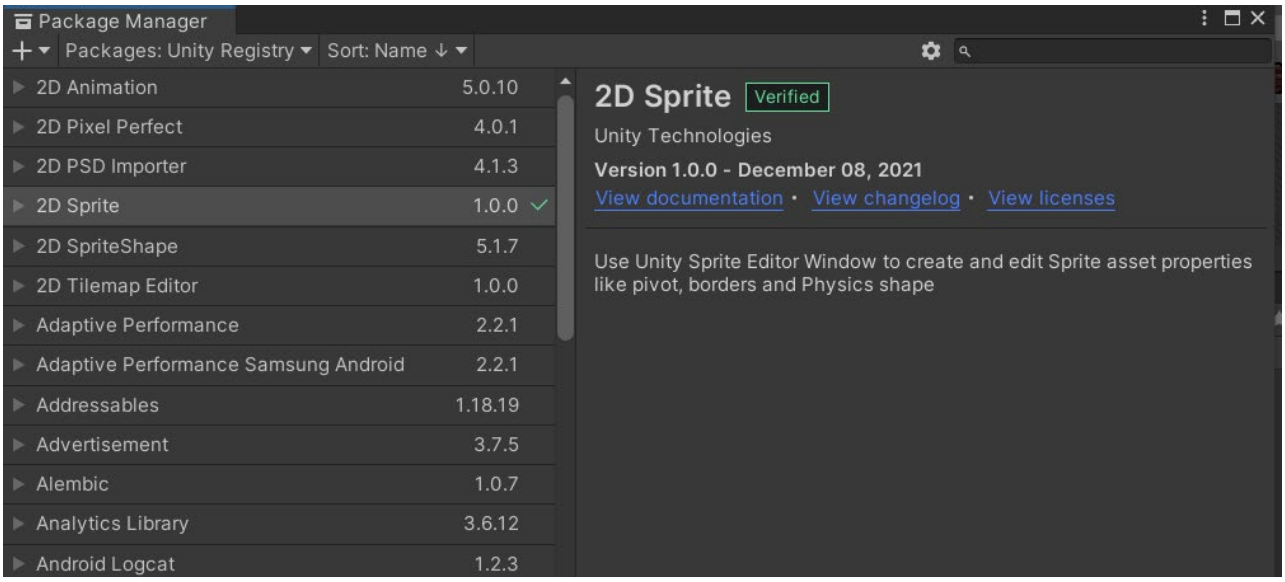


estas imágenes podremos arrastrarlas a la escena del juego. Un spritesheet tiene que tener sus dimensiones (ancho x alto) en potencias de 2, en este caso la imagen es de 1024 x 256.

Crea la **carpeta Sprites** dentro de Asset y arrastra dentro de ella el siguiente archivo. Cambia la **propiedad Texture Type a Sprite (2D and UI) y la propiedad Sprite Mode de Single a Multiple** ya que contiene varias imágenes. Una vez hecho los cambios, pulsa en Apply.

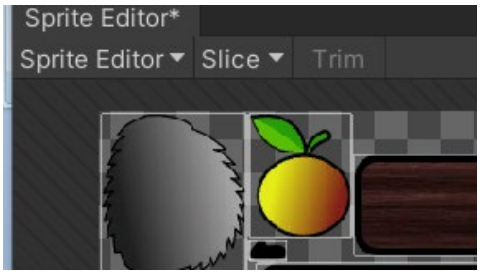


NOTA: Si no tienes instalado el Sprite Editor o cualquier otro paquete de Unity hay que ir a la opción Windows > Package Manager y en la siguiente ventana en la pestaña Packages: Unity Registry y seleccionas 2D Sprite e instalas.



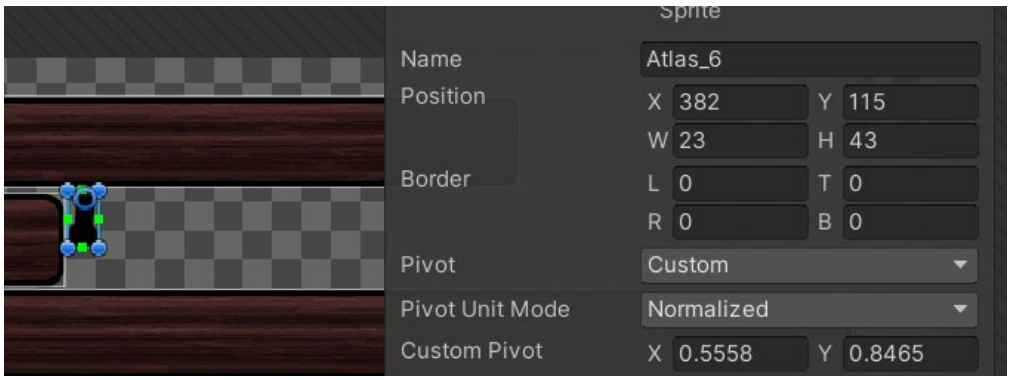
Para extraer cada elemento de la spriteSheet por separado debemos editarlo pulsando en el botón Sprite Editor. Realiza las siguientes acciones en el Sprite Editor:

- Utiliza la herramienta automática para recortar todas las imágenes **(Slice/Type=Automatic)**. Aparecerá un cada objeto de la imagen en un recuadro diferente.



- Prueba el uso de la herramienta **Trim** para ajustar los bordes de las cajas a las imágenes.
- Modifica el punto de rotación de los elementos. Es el punto desde el que cada imagen va a rotar cuando hagamos las animaciones. (Por defecto están en Pivot: Center)

  En el siguiente elemento hemos cambiado el punto de rotación:



- Puedes dar un nombre a cada caja:



- Utiliza el botón Apply para guardar los cambios.

- Cierra el Sprite Editor y revisa lo que ha pasado en la carpeta Sprites. Verás que se ha creado cada objeto por separado.

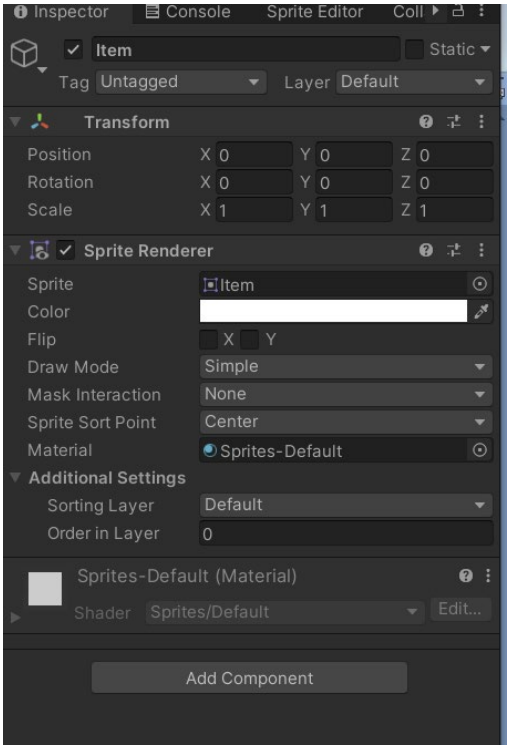
Si quieres tener los mismos nombres y puntos de rotación para seguir este material, borrar tu SpriteSheet y arrastra los dos ficheros a la vez de este enlace.

4.- Prefabs

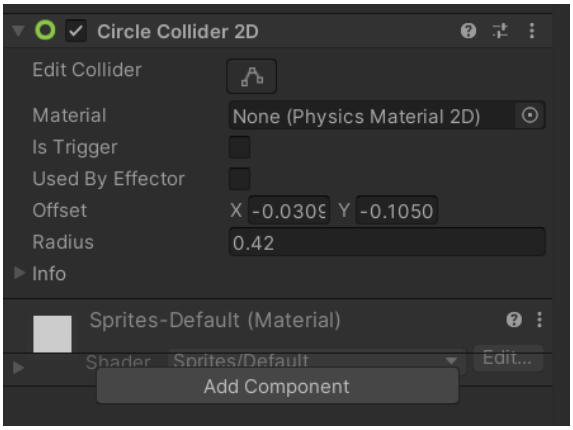
Los prefabs son sprites o conjuntos de sprites con unas propiedades predeterminadas que se utilizan como plantillas para luego añadir a la escena. Esta plantilla se puede reutilizar en el desarrollo del juego. Si se modifica alguna propiedad del prefab, todos los objetos que se han creado a partir de él también se modifican.

Vamos a crear un prefab para el sprite Item (manzana):

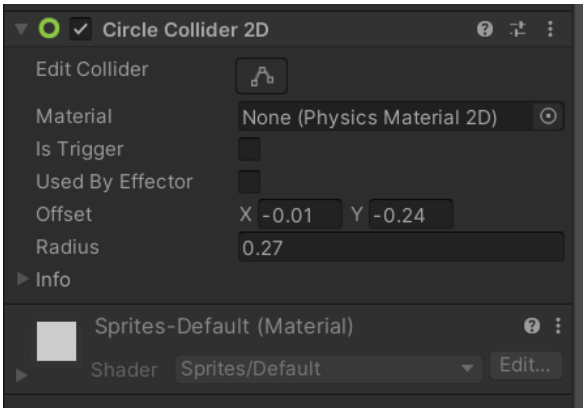
- Crea una carpeta Prefabs en el proyecto y dentro de ella crea el prefab item.
- Añade a la escena un sprite Item y pon sus coordenadas en 0, 0, 0.



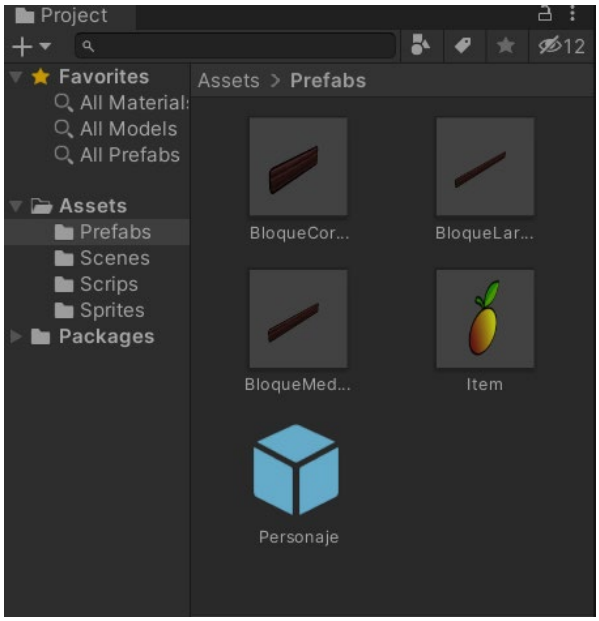
- Hay que añadir un Collider para que sea un detector de colisiones. Para ello en el Item de la escena en el inspector pinchamos el botón añadir componente y seleccionamos Phisycs 2D > Circle Collider “D y nos muestra lo siguiente:



Ajustamos el radio y las coordenadas al ítem, para este caso los valores serían:



- Arrástralo de la escena a la carpeta de prefabs. Así se crea un prefab.



Las modificaciones de un prefab se pueden hacer directamente sobre el prefab o con un objeto de la escena que se haya creado con el prefab.

Añade colliders (Box Collaider 2D) y crea prefabs de todos los bloques que se muestran en la última imagen.

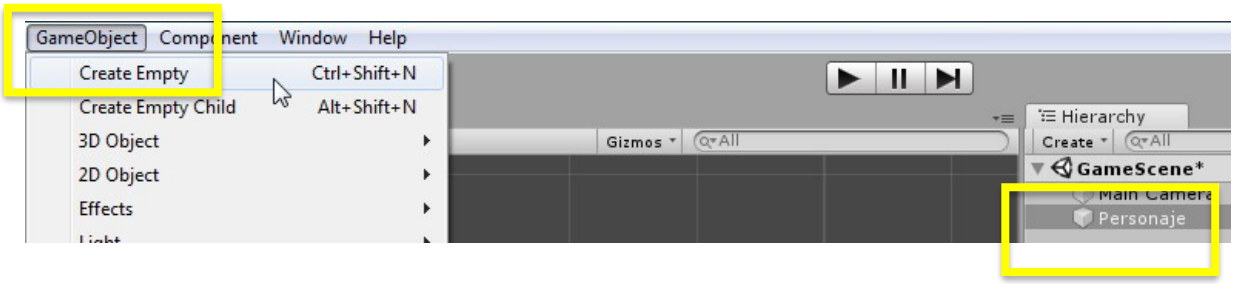
5.- Personaje

Vamos a montar el personaje del juego. El objetivo es añadir todos los elementos del mismo a la escena hasta componerlo. Luego crearemos el prefab Personaje.

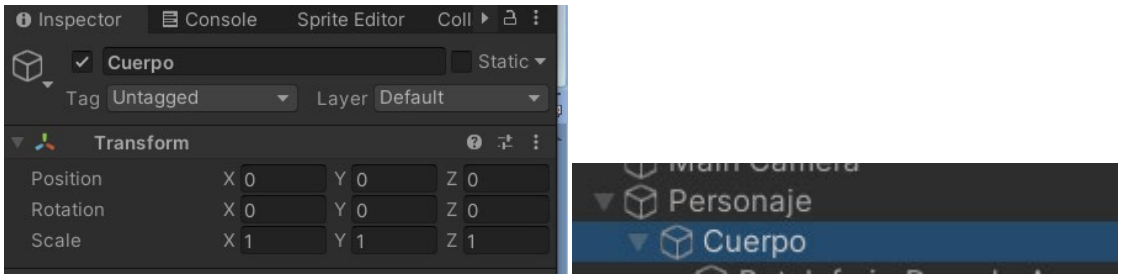


MONTAR EL PERSONAJE

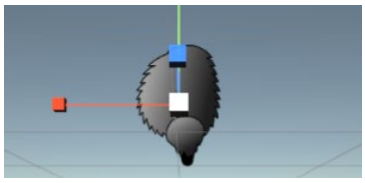
- Accede al menú Game Object/Create Empty.
- Se crea un nuevo objeto en la escena. Cambia su nombre a Personaje.



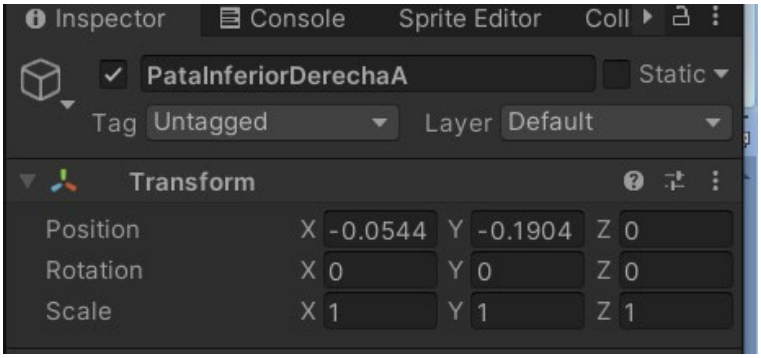
- Arrastrar el cuerpo a la escena y meterlo dentro de personaje:



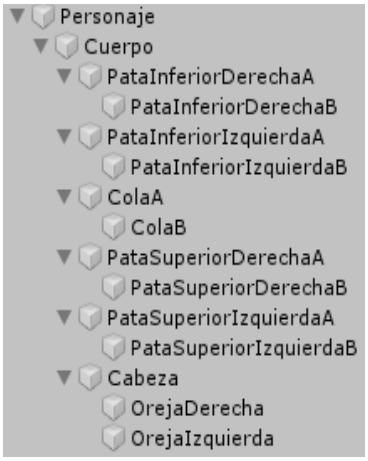
- Arrastrar la pata inferior y poner los siguientes datos:



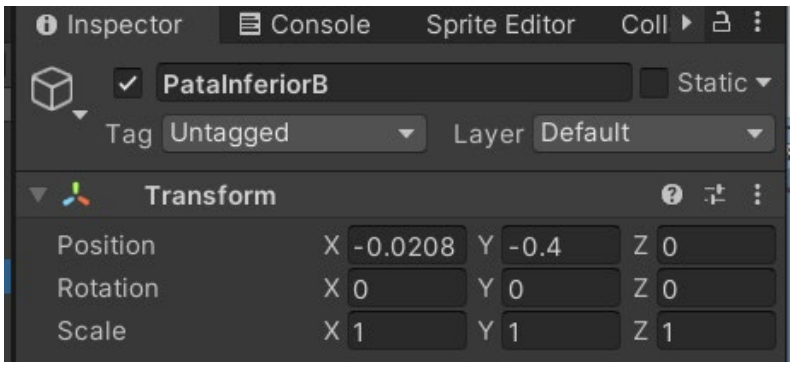




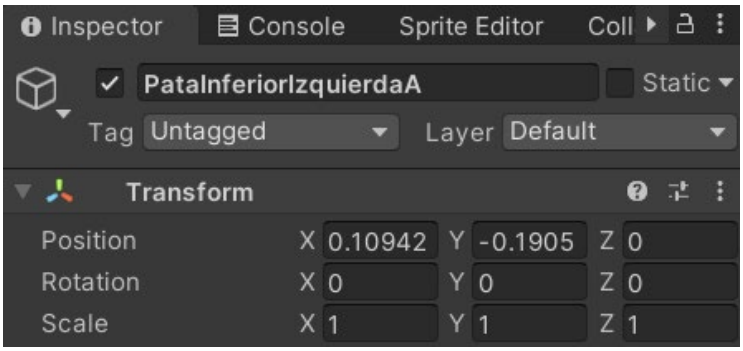
- El árbol que hay generar es el siguiente:



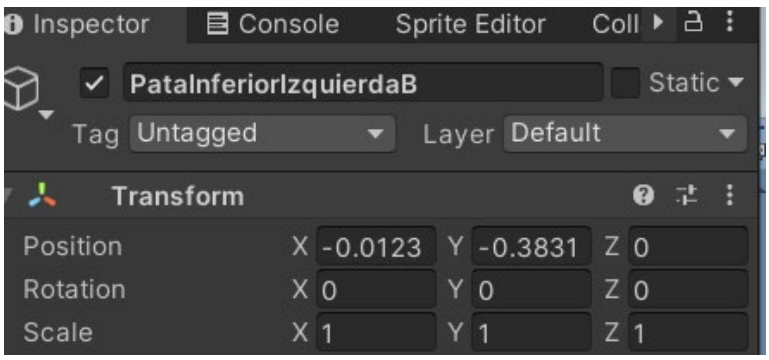
- Datos PataInferiorDerechaB:



- Datos de PataInferiorIzquierdaA:



- Datos de PataInferiorIzquierdaB:

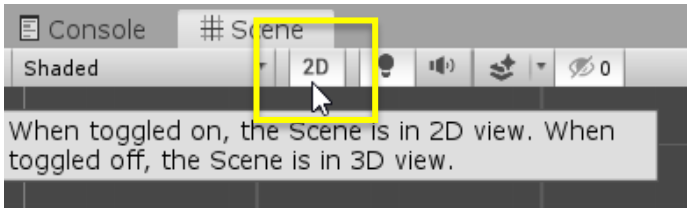


- Para colocar los objetos utiliza la barra de herramientas.

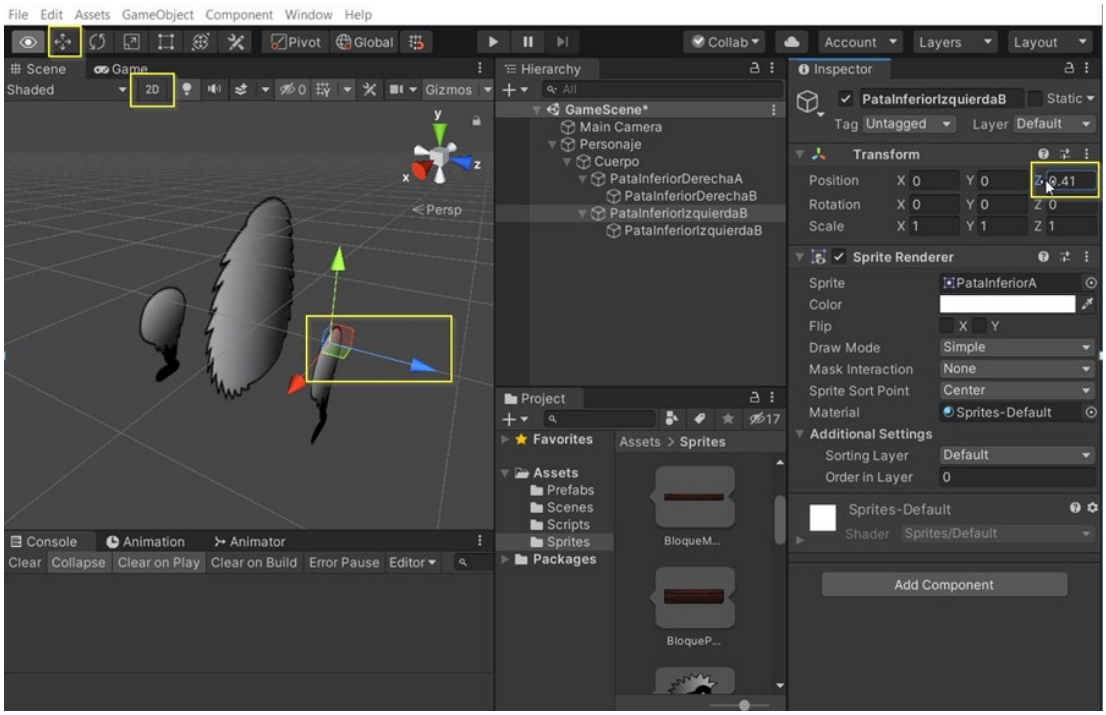


- Es posible duplicar elementos. Hay que seleccionarlo y pulsar **ctrl+d**. Cuando tengas la pata derecha A, puedes duplicarla y cambiar el nombre para la pata izquierda B.
- Para hacer que las patas no se superpongan vamos a cambiar la posición en el eje Z del elemento.

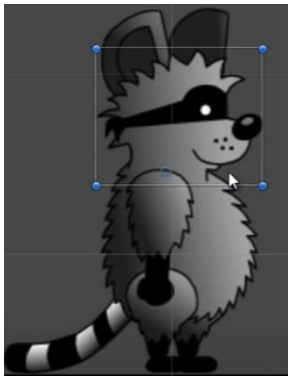
- Activa el modo 3D:



- Observa cómo queda el personaje en el modo 3D. No te preocupes por la separación que aparece entre los objetos no se ve en 2D. Para cambiar la posición del eje Z, con la herramienta de mover seleccionada, puedes mover la flecha azul del eje Z o bien, modificar el valor de la coordenada Z en las propiedades del objeto.

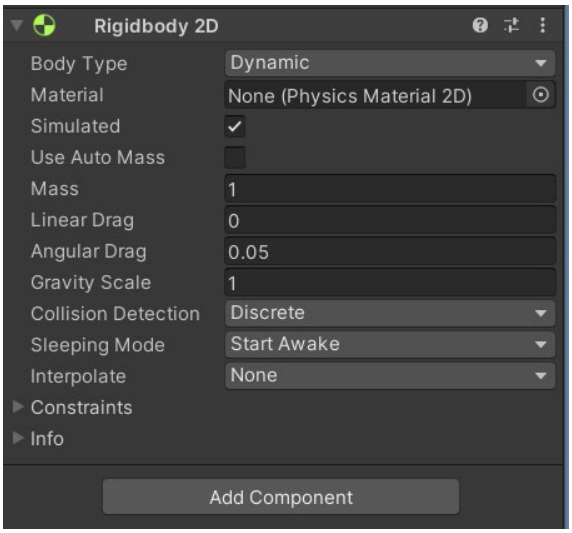


- Y así con todos los componentes hasta formar el personaje:

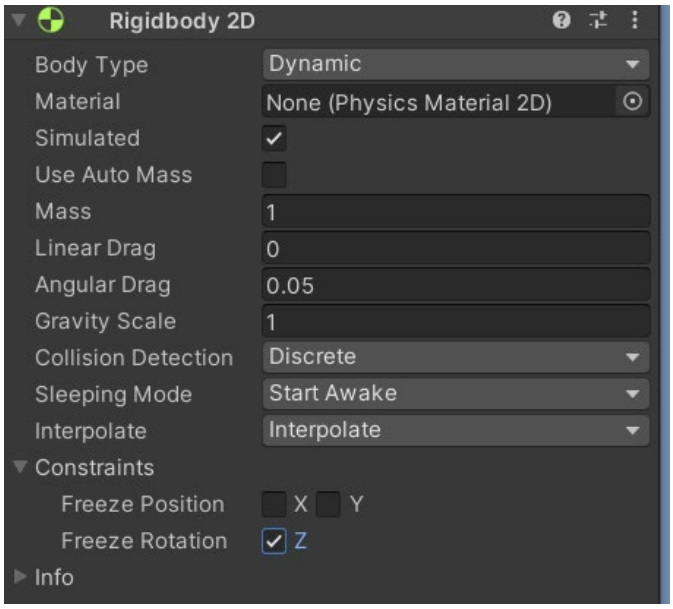


DEFINIR LA FÍSICA DEL PERSONAJE

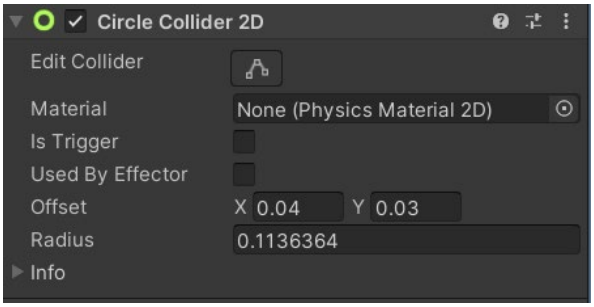
- Al personaje creado le debe de afectar la física: gravedad, velocidad, etc.
- Para poder configurar la física del personaje tenemos que añadir el componente RigidBody2D.



- Lo vamos a configurar de la siguiente manera: se bloquea la rotación en el eje Z para que el personaje no gire cuando se mueva y se recomienda activar la interpolación en el RigidBody del personaje que controla el jugador y desactivarla en los demás.



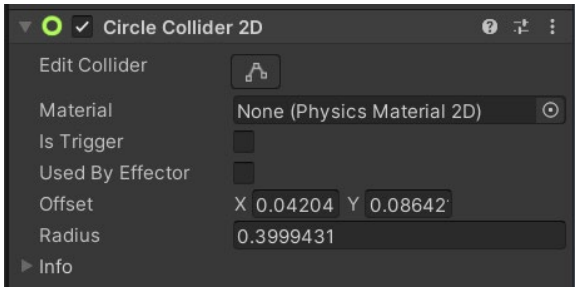
- Añade a la escena un bloque y colócalo debajo del personaje. Si ejecutas el juego observarás que ahora el personaje se cae de la escena porque le afecta la gravedad.
- Esto se soluciona añadiendo un Circle Collider a las dos patas inferiores.

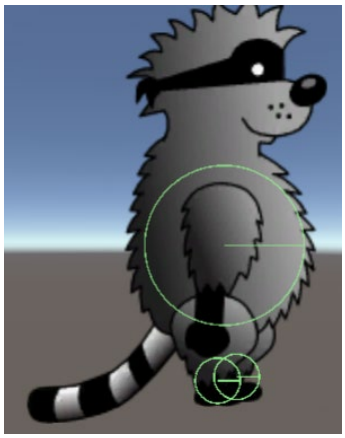




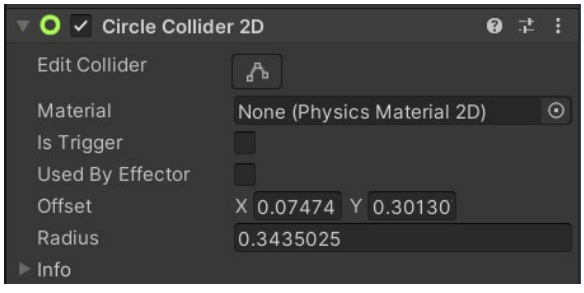
- También añadimos dos Colliders más, uno al cuerpo y otro a la cabeza. Lo hacemos para que el personaje pueda interactuar con el resto de elementos del juego cuando choque con ellos: Recoger items, chocar contra los bloques, etc.…

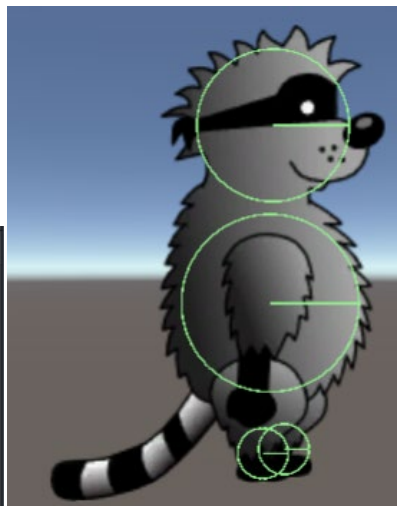
- El Collider del cuerpo tiene la siguiente configuración:





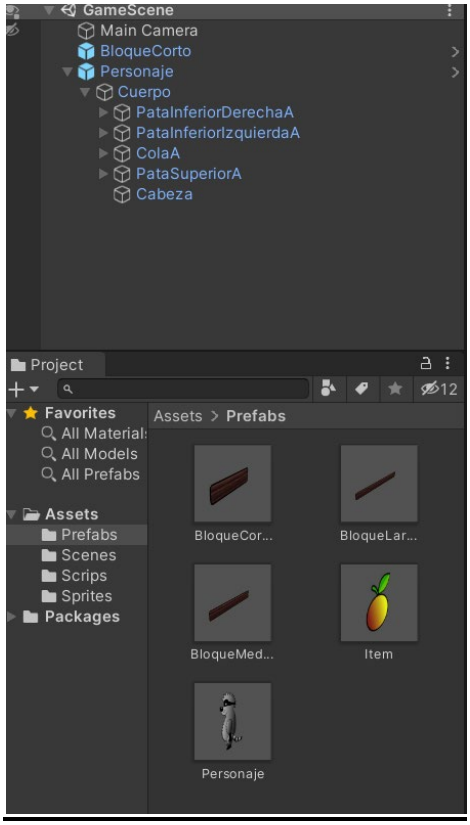
- El Collider de la cabeza tiene la siguiente configuración:





CREAR PREFAB PERSONAJE

- Creamos el prefab Personaje arrastrándolo a la carpeta Prefabs.



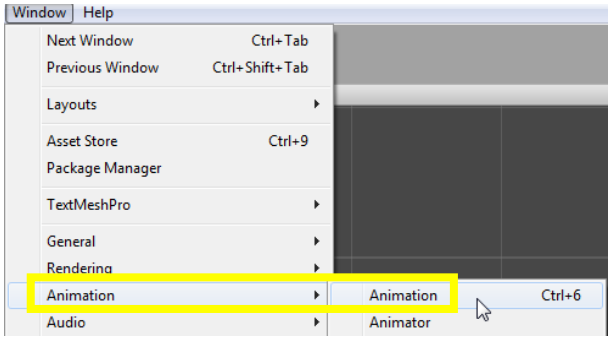
6.- Animaciones

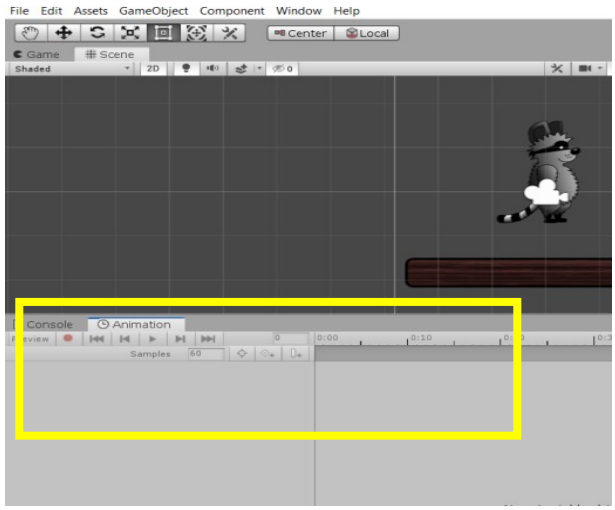
Las animaciones nos permiten hacer que los componentes del personaje se muevan para darle dinamismo.

Vamos a hacer tres animaciones, dependiendo del estado en que se pueda encontrar el personaje:

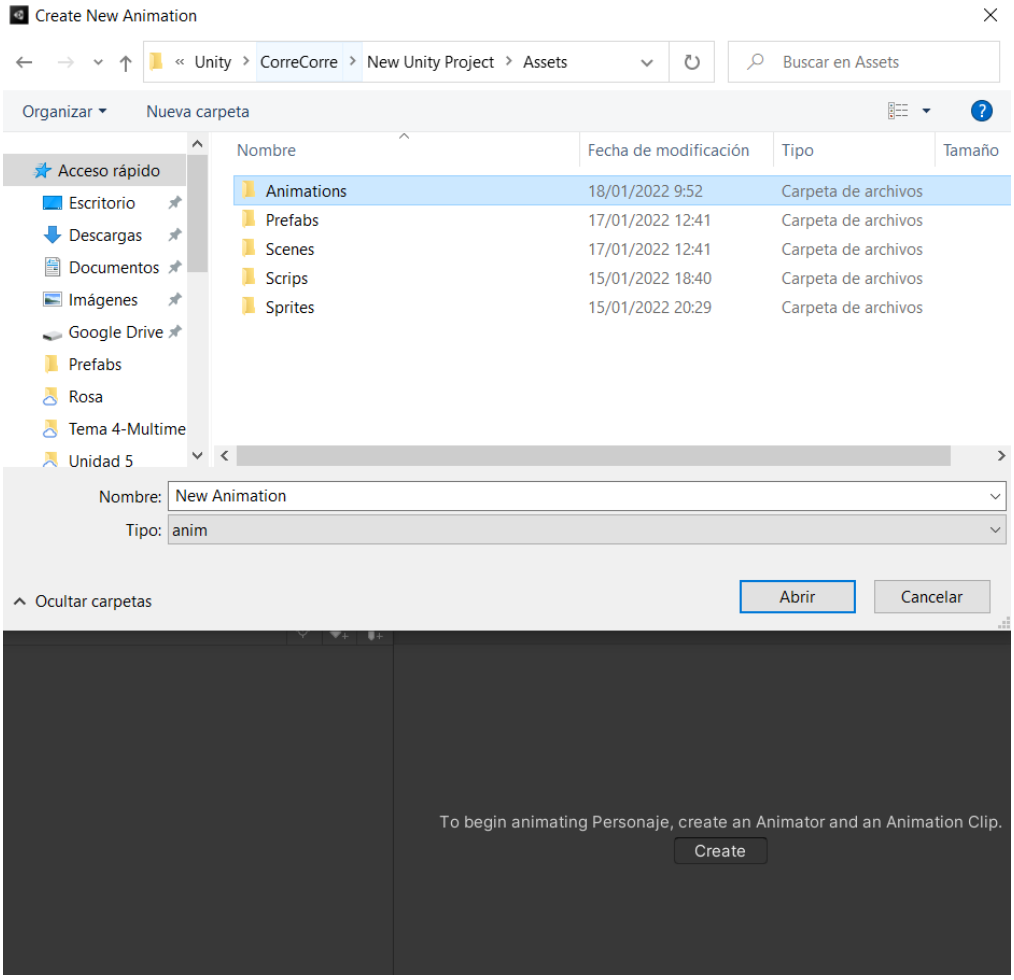
- Parado
- Corriendo
- Saltando

Una vez hechas las animaciones, el siguiente paso será configurar el paso de una animación a otra.
Para hacer una animación añadimos a nuestro proyecto la ventana de animaciones y la anclamos a la parte inferior de la ventana:

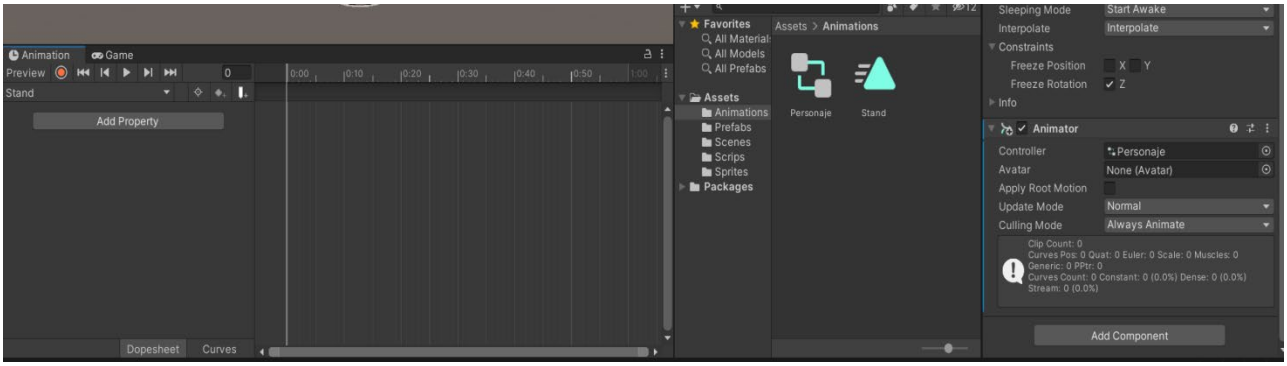




En la ventana Animation damos al botón Create para crear un nuevo clip para el personaje. Nos muestra la siguiente ventana donde en la carpeta Asset creamos otra carpeta llamada Animations donde se guardarán las animaciones del proyecto, la primera que vamos hacer la llamaremos Stand (estar de pie, parado).

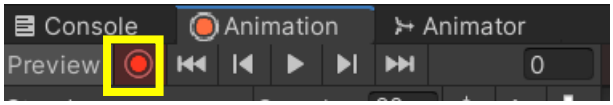


Se puede observar que al personaje se le ha añadido automáticamente el componente Animator.



Hacer una animación consiste en ir variando alguna de las propiedades del personaje a lo largo del tiempo.

CUIDADO: Para hacer una animación, asegurarse de que el botón de grabación está activo. De esta forma puedes modificar cualquier propiedad del objeto y se modificará solamente en la animación, no en el objeto en sí mismo.



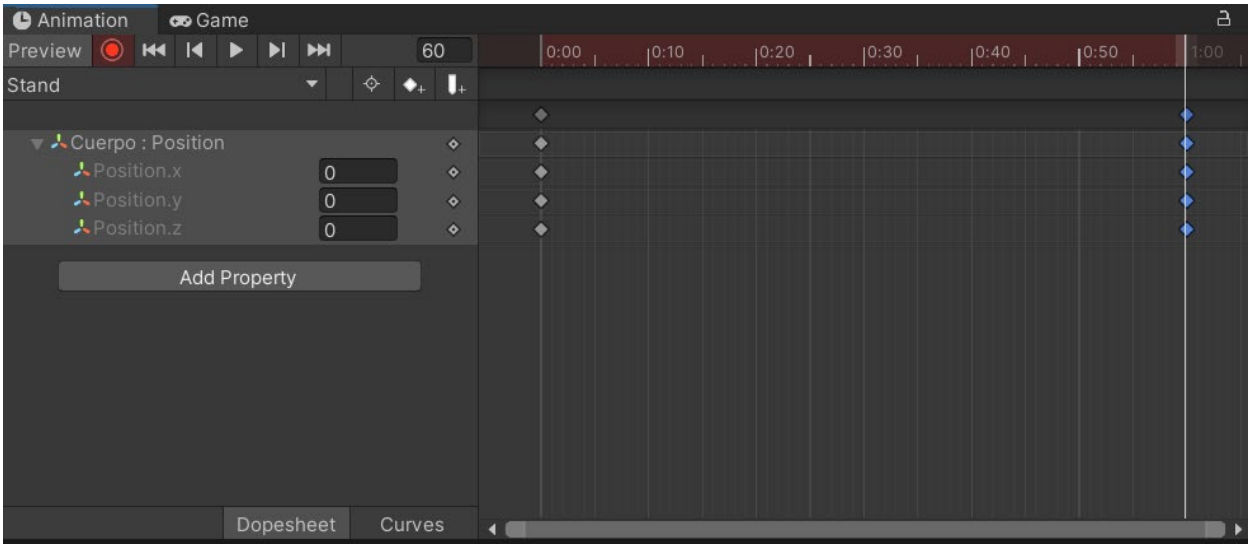
ANIMACIÓN STAND (PARADO)

Esta animación consiste en hacer que el personaje esté respirando.

Va a tener 3 estados, uno en el fotograma 0, otro en el 0:30 y otro en el 1. El primer y tercer fotograma son iguales y corresponden con el personaje tal y como está definido. Lo normal en una animación es que el primer y último fotograma sean iguales.

1. **Activa la grabación**.

2. **Creación del fotograma 0s y 1s**: Nos ponemos en el instante 0 y añadimos la propiedad en botón Add Property Cuerpo/Transform/Position. Verás que se crean el fotograma en 0 y en 1, con los mismos valores para la posición den cuerpo en x, y, z. No modificaremos nada ya que en estos fotogramas el personaje coincide con el que está definido.



3. **Creación del fotograma 0:30s:** Nos situamos en el segundo 0:30 y creamos un fotograma en este punto haciendo clic en él. Solamente modificaremos algunas propiedades de los componentes del personaje en el fotograma 0:30. Como se dice anteriormente, la forma más cómoda de modificar propiedades es seleccionando el objeto a modificar y en la escena realizar la modificación con las opciones de la barra de herramientas. , pero asegúrate siempre de que estás grabando.

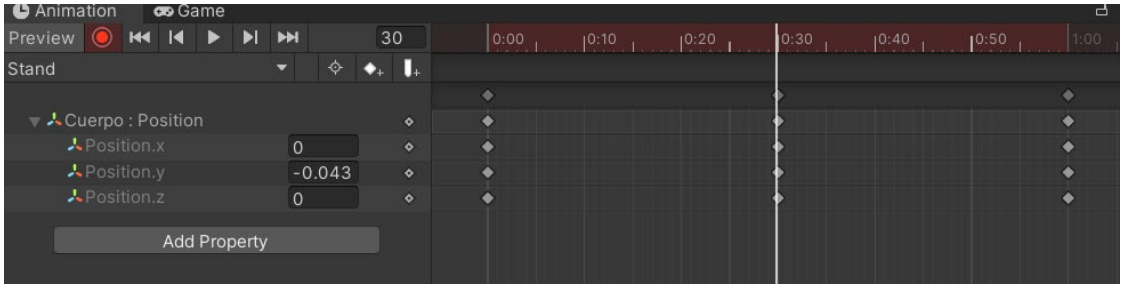


Puedes comprobar cómo queda la animación haciendo clic en Play Animation Clip:

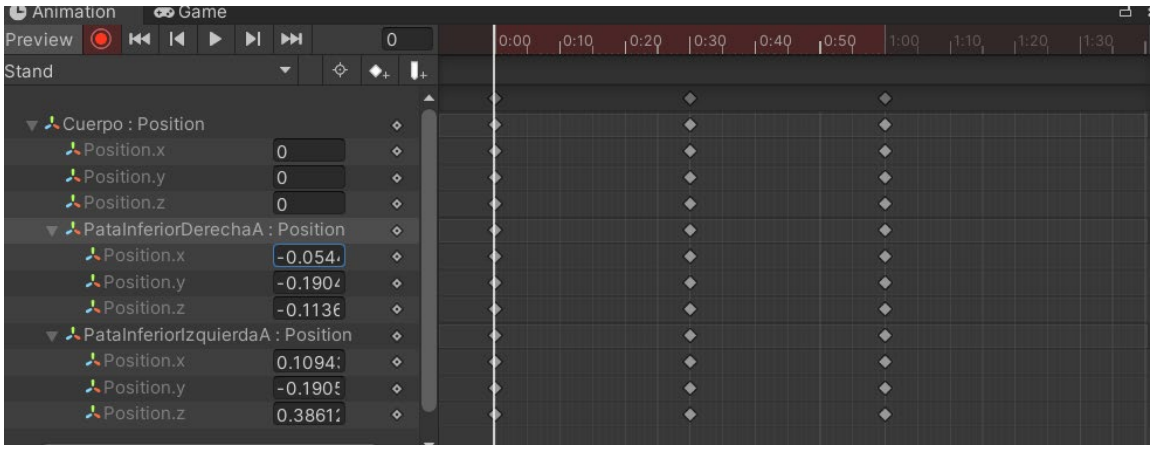


Lo que vamos a modificar en este fotograma es:

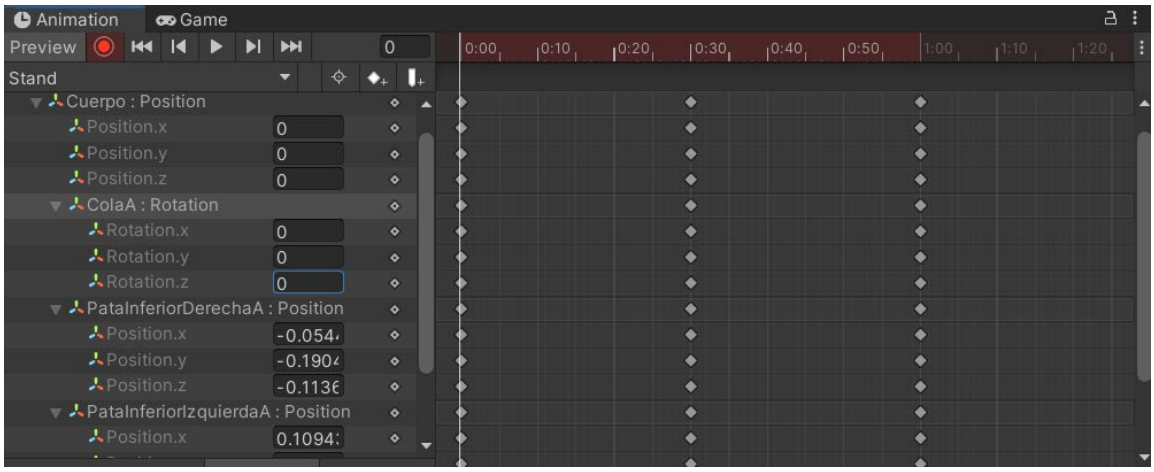
- **Bajar el cuerpo**: Como ya aparece la propiedad Transform/Position del cuerpo, solamente hay que modificar el valor de **y** del cuerpo.



- Mantener la altura de las **patas inferiores**: sitúate en el fotograma 0:30 y modifica el valor de **y** para que suban un poco o sube la altura con la herramienta mover.

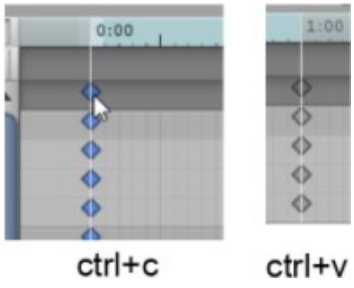


- **Rotar la cola:** Procede de igual forma para añadir la cola. La propiedad a modificar es Transform/Rotation. Modificamos el valor de la componente z para hacer la rotación o haz la rotación con la herramienta rotar.



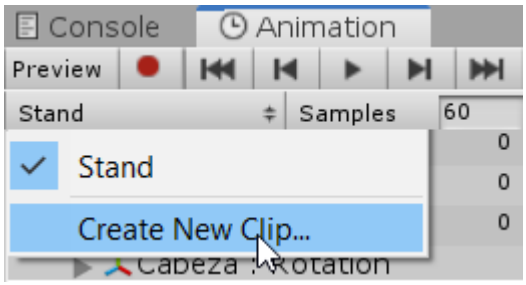
- **Subir la pata superior derecha:** Procedemos igual que con las patas inferiores (Posición Y).
- **Rotar la cabeza hacia adelante:** Procedemos igual que con la cola (Rotación).
- **Rotar las orejas:** Procedemos igual que con la cola (Rotación).

Es posible copiar y pegar fotogramas (ctrl+c/ctrl+v). Hay que situarse en el estado a copiar y hacer clic sobre el rombo más alto y copiar; a continuación, nos ponemos en el fotograma en el que queramos pegar, y pegamos.



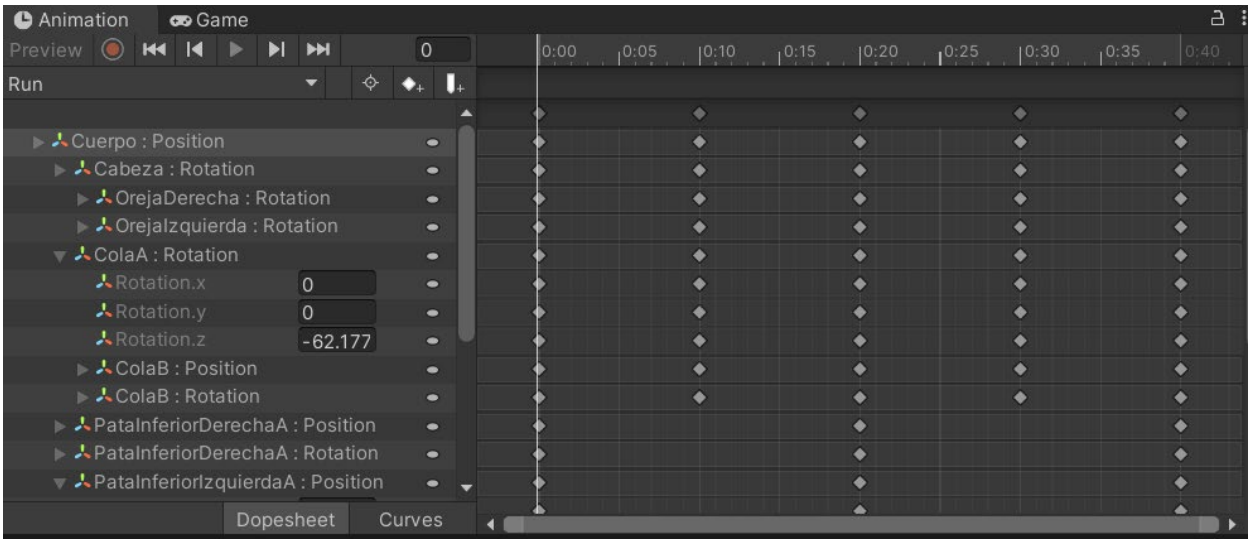
ANIMACIÓN RUN (CORRER)

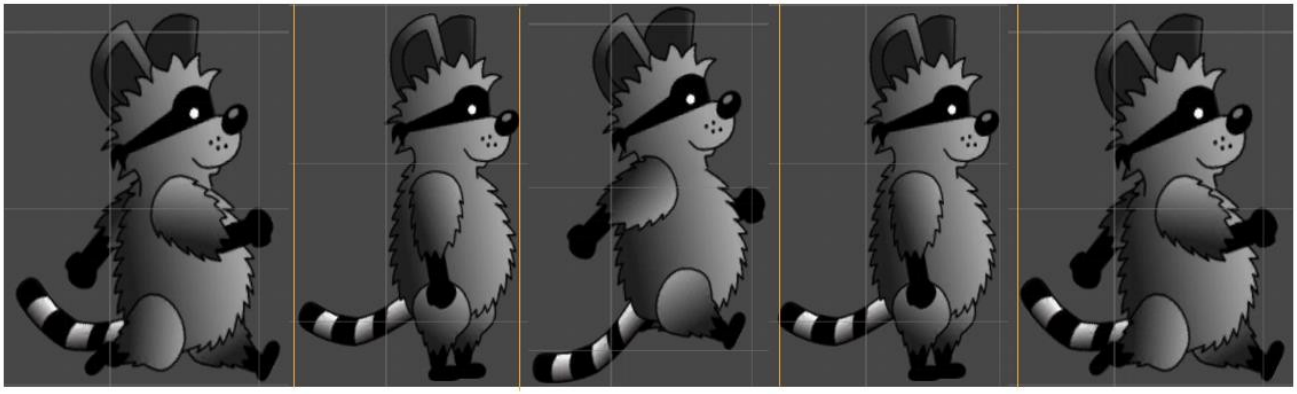
Selecciona el personaje y en la pestaña de Animation >Stand>Create New Clip y llámalo Run.



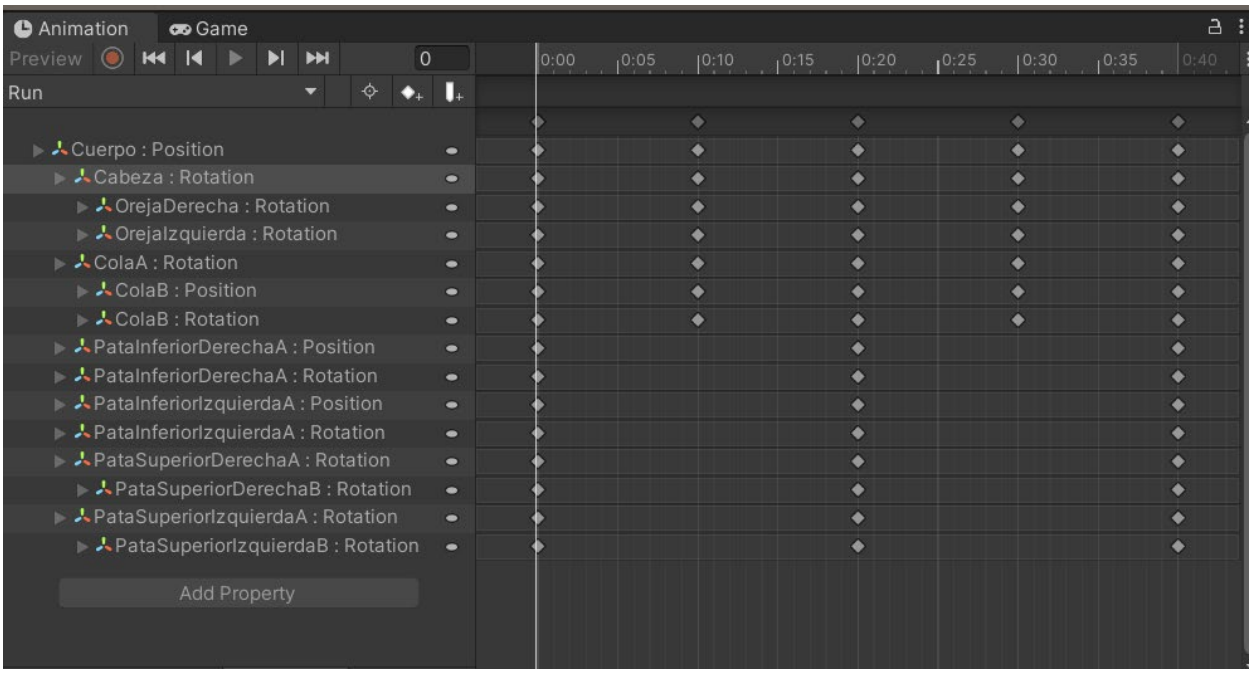


En esta animación vamos a distribuir el tiempo en 5 bloques:



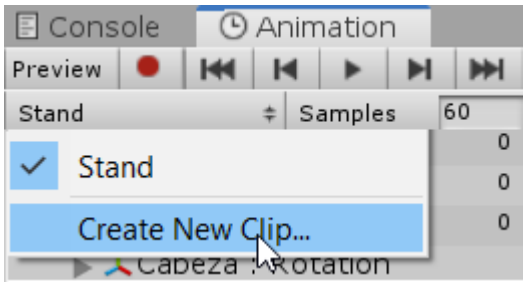


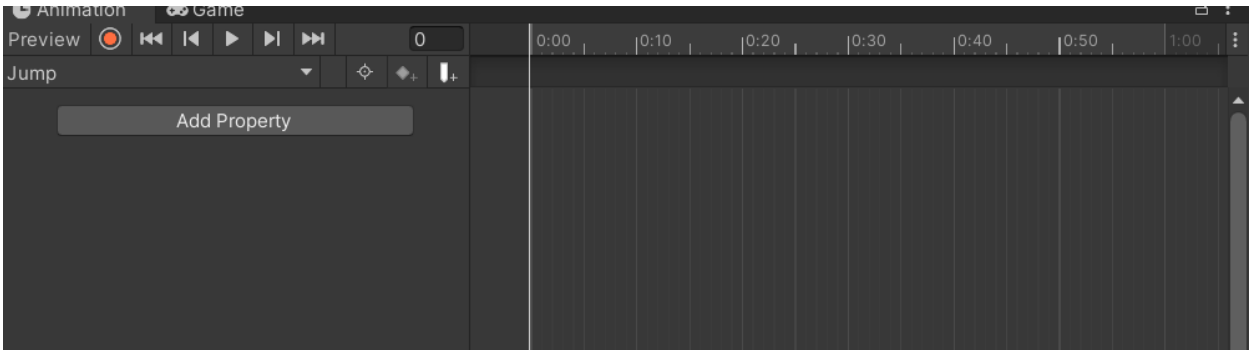
- En los instantes 1, 3 y 5, el personaje está desplazados en el eje Y hacia arriba.
- Debes ajustar las propiedades necesarias de las diferentes partes del cuerpo para obtener un resultado parecido al de la imagen.



ANIMACIÓN JUMP (SALTAR)

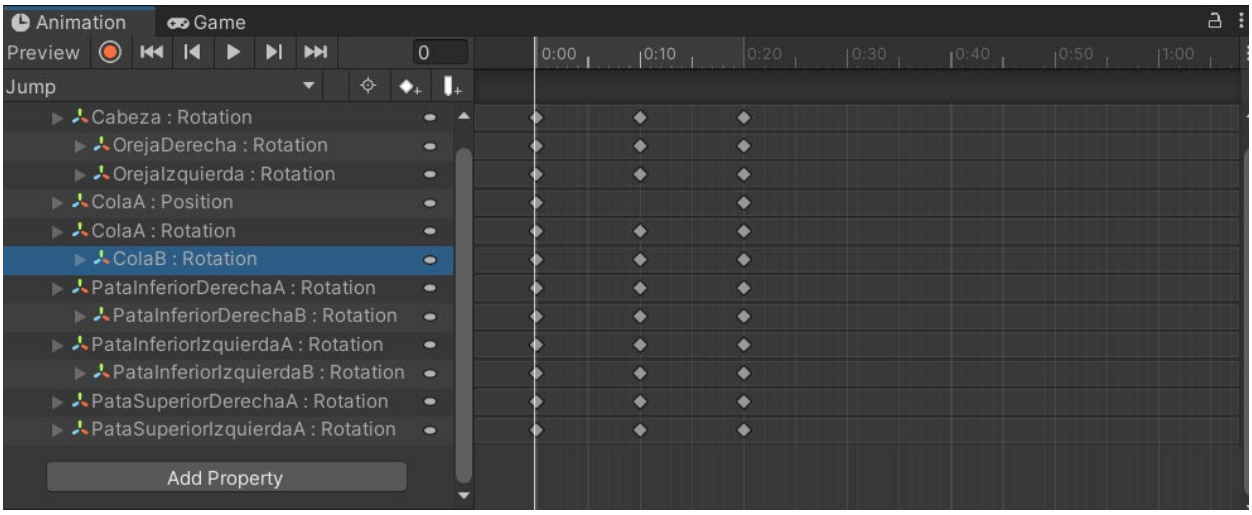
Selecciona el personaje y en la pestaña de Animation >Stand>Create New Clip y llámalo Jump.





En esta animación vamos a distribuir el tiempo en 3 bloques tal y como muestra la imagen:

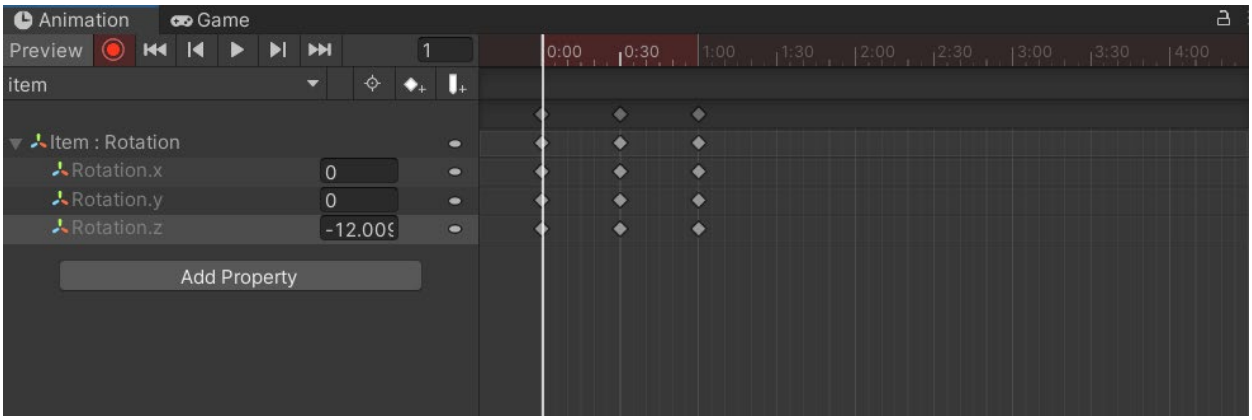




ANIMACIÓN ITEM (MANZANA)

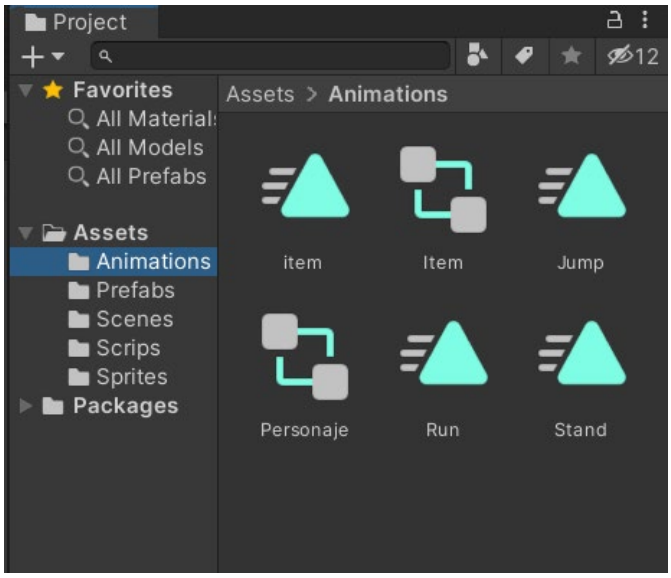
Se añade un item a la escena para darle la animación. Y a continuación, en la pestaña de Animaton creamos un nuevo clip que se llama Item.amin.

En esta animación vamos a distribuir el tiempo en 3 bloques tal y como muestra la imagen:

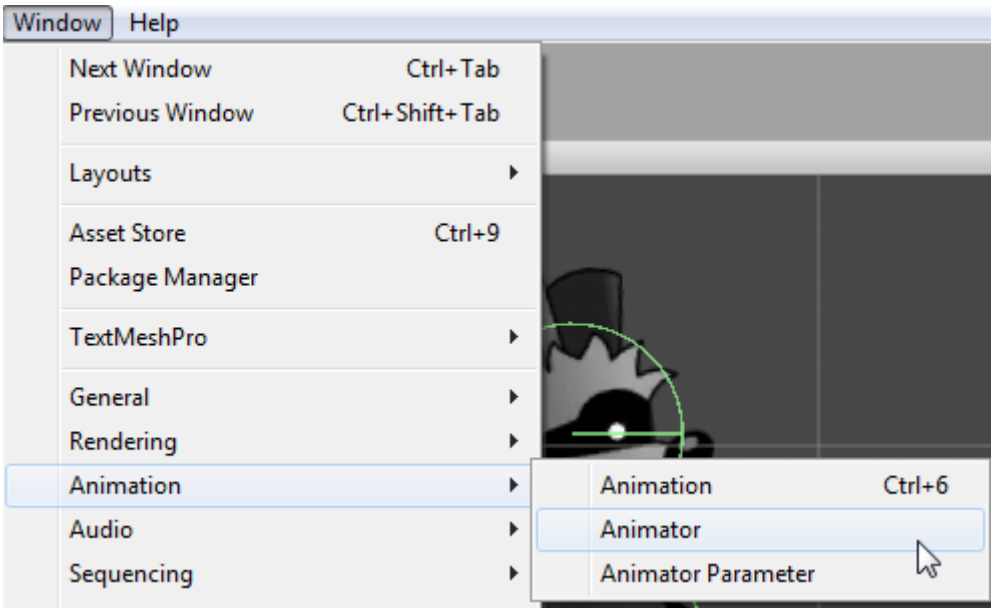


7.- Componente Animator

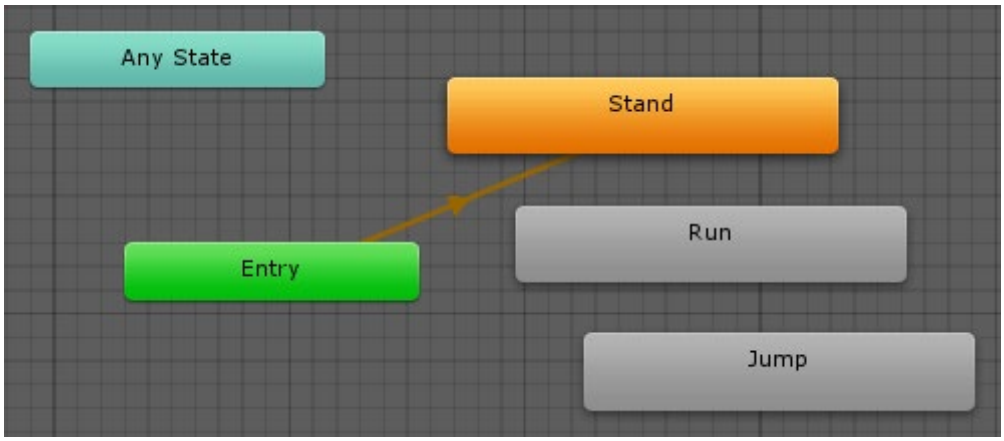
Todas las animaciones y el componente animator se han creado en la carpeta Asset/Animations.



El componente Animator permite gestionar el paso de una animación a otra. Para visualizar este componente accede al menú **Window/Animator.**



Inicialmente aparecen todas las animaciones que hemos hecho para el personaje, pero no hay conexión entre ellas todavía. La de color naranja es la inicial que se ejecuta por defecto. En este caso, cuando se inicie el juego, el personaje va a estar en la animación Stand: Ejecuta el juego para comprobarlo.



TRANSICIONES

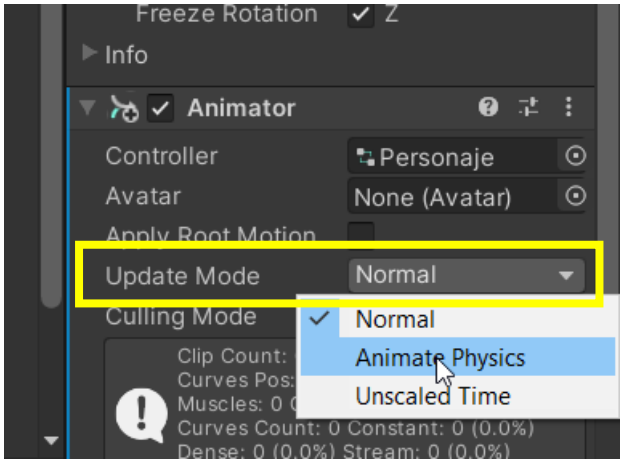
Una transición es el paso de una animación a otra.

En nuestro ejemplo tendremos que definir las siguientes transiciones:

- **De Stand a Run:** El personaje comienza el juego en Stand, pero más adelante haremos que empiece a correr cuando hagamos clic en el botón izquierdo del ratón.
- **De Run a Stand:** Si el personaje se detiene, pasa a la animación Stand.
- **De Stand a Jump:** El personaje parado, saltará cuando se haga clic con el botón izquierdo del ratón.
- **De Jump a Stand:** Después de saltar, el personaje sigue en Stand.

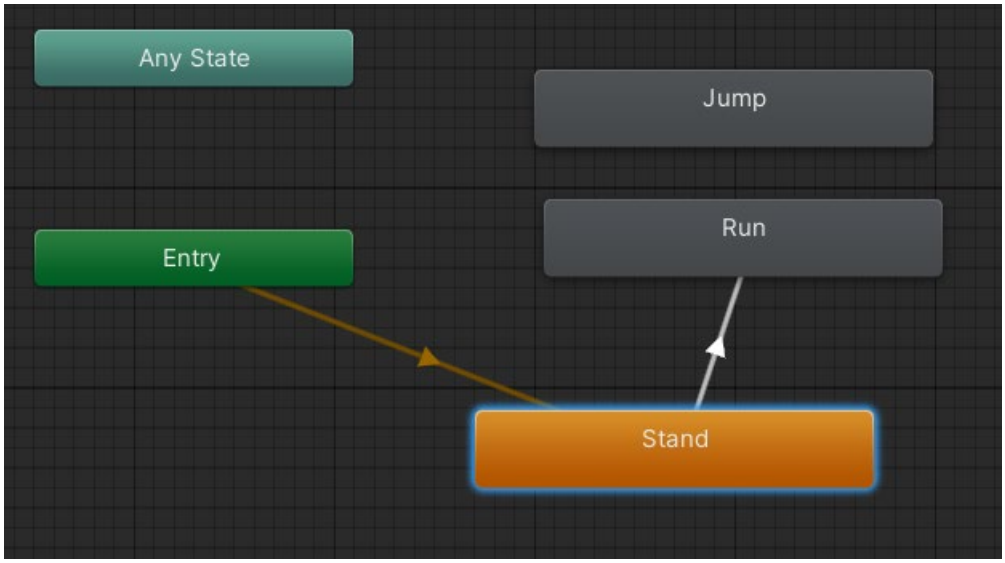
- **De Run a Jump:** Una vez que el personaje empieza a correr, programaremos que pueda saltar cuando se haga clic con el botón izquierdo del ratón.
- **De Jump a Run**: Después de saltar, el personaje sigue corriendo.

Para que las animaciones se sincronicen con la física de la escena debemos modificar en el componente Animator del personaje la propiedad Update Mode y asignarle el valor Animate Physics.



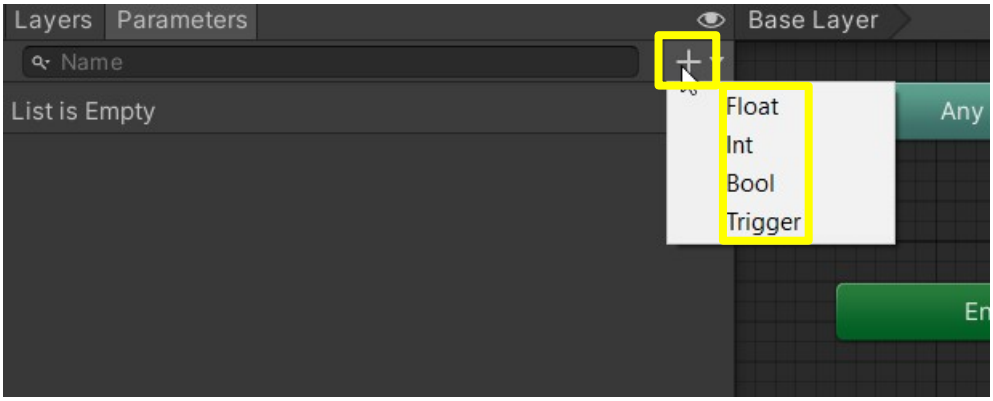
Transición de Stand a Run

Pulsa el botón derecho sobre el estado Stand y selecciona Make Transition. Haz clic sobre el estado Run. Aparecerá una flecha que representa la transición:



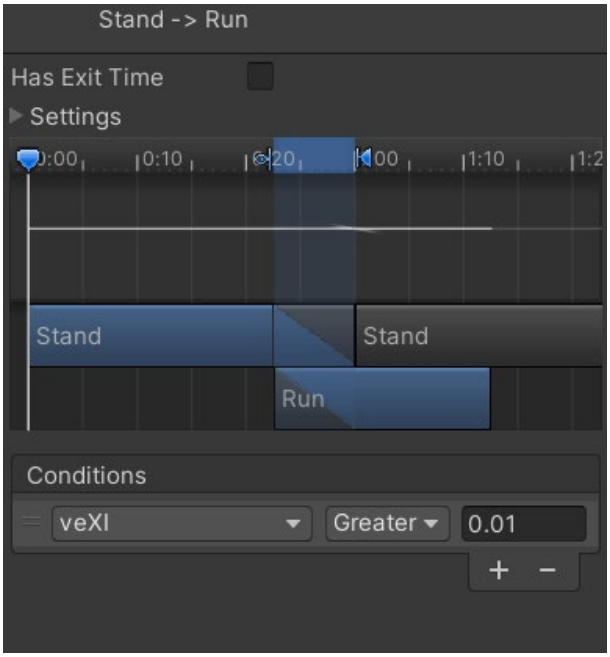
Las transiciones permiten **definir variables** que controlen cuando se activa la transición en función del valor de la misma. El valor de las variables se cambiará a través de código, cosa que veremos más adelante.

Para **añadir una variable** vamos a la pestaña **Parameters** del componente Animator.



Añade una variable velX de tipo float. Por defecto el valor será 0.

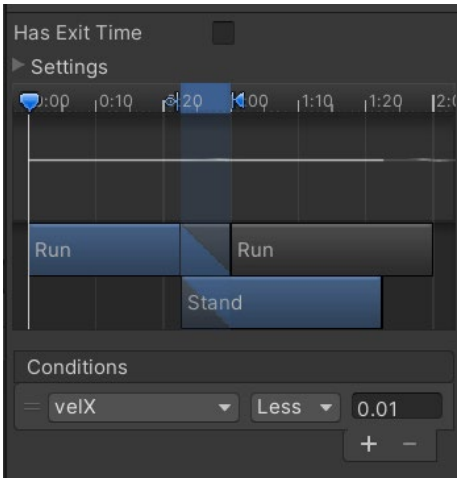
Crearemos una condición para que cuando la **velX** sea **mayor que 0.01**, se realice la transición de Stand a Run. Para crear esta condición, selecciona la transición de Stand a Run. En el Inspector de propiedades, crearemos la condición y desmarcaremos la opción **Has exit** time para evitar un retardo indeseable en la transición:



Una vez puesta esta condición, se realizará el paso de Stand a Run cuando desde cualquier punto del juego, por código, se modifique el valor de la variable velX y este sea mayor que 0,01.

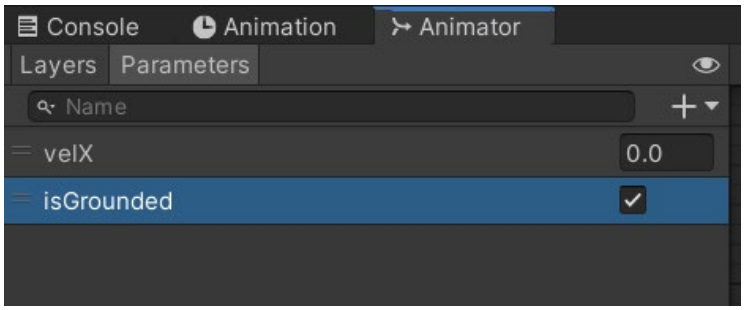
Transición de Run a Stand

Crea la transición y establece la condición apropiada para la transición de corriendo ha parado (Lo contrario a la transición anterior):

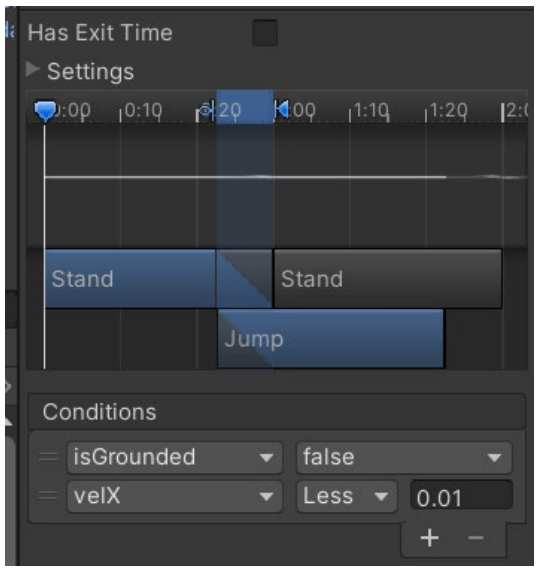


Transición de Stand/Jump

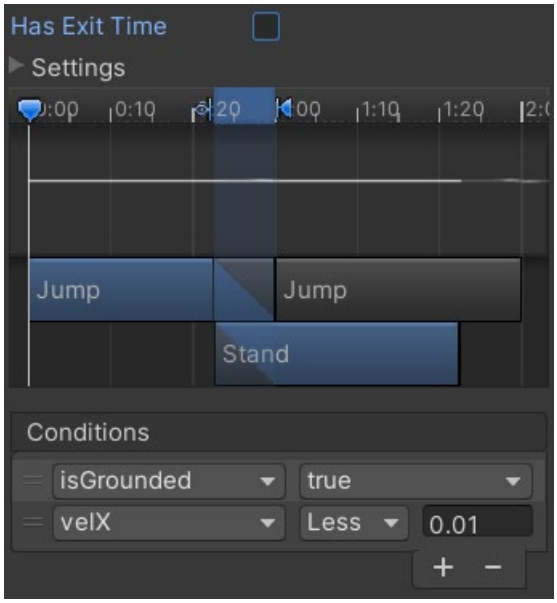
Para la transición de parado a saltando necesitamos una **variable booleana** que nos diga si el personaje está o no tocando el suelo. Añadimos un parámetro booleano **isGrounded** que por **defecto a true**.



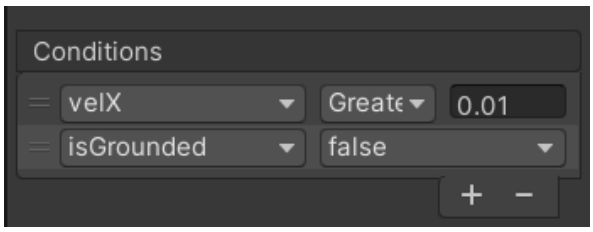
Esta transición se produce si el personaje está parado y no está tocando el suelo. Añadimos a la transición su condición:



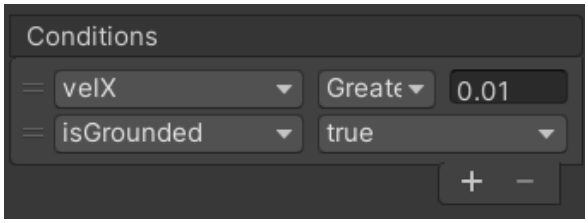
Transición de Jump/Stand



Transición de Run/Jump



Transición de Jump/Run



8.- Programación Saltar

El objetivo de este apartado es hacer que el personaje salte cuando el usuario toque la pantalla o haga clic izquierdo en el ratón.

Para añadir comportamientos a un objeto, necesitamos un script que implemente esa funcionalidad.

En nuestro caso, el **personaje debe tener un script que capture las pulsaciones** y le dé un empujón hacia arriba. Crea un archivo en la carpeta de Scripts de nombre **ControladorPersonaje**.

Se abre para editarlo y las **partes del script son: Clase, Método Start, método Update:**

- **Clase:** *Muy importante, a tener siempre en cuenta, el nombre de la clase tiene que coincidir con el nombre que el fichero .cs. Mucho cuidado si renombráis algún script.*
- **Start:** Se ejecuta una sola vez al inicio del script. Útil para realizar inicializaciones.
- **Update:** Se ejecuta una vez en cada fotograma. Se trata por tanto del "bucle infinito" del script. Se utiliza en el control de los objetos.

Programando el salto

Lo primero vamos a declarar la variable **fuerzaSalto**. Con ella podemos variar la potencia del salto del personaje. Por defecto le damos valor 100.

Esta variable la declaramos como atributo público de la clase, de esta forma podremos darle un valor desde el inspector de propiedades del personaje, cuando le asignemos este script.

```
public class ControladorPersonaje : MonoBehaviour
{
    public float fuerzaSalto=100f;

        // Start is called before the first frame update
    void Start()
```

Guarda el script y vuelve a Unity. Aplica el script al personaje, arrastrándolo y verás que en el inspector de propiedades del personaje, aparece fuerzaSalto y nos permite modificar su valor. Esto es así porque hemos declarado la variable pública y podemos modificar su valor sin modificar el código en el script.



Volvemos al script para añadir la detección, tanto el toque de la pantalla, como la pulsación del botón izquierdo del ratón. Para ello usamos el método **Input.GetMouseButtonDown(0).** El 0 indica el botón izquierdo. Esta función detecta el botón izquierdo del ratón en un ordenador y el toque de pantalla en un móvil.

```
void Update()
{
    //Detección de pulsación en la pantalla o clic izquierdo del ratón
    if (Input.GetMouseButtonDown(0))
    {

    }

}
```

Dentro de este if tenemos que hacer que el personaje salte. Esto implica que tenemos desplazarlo hacia arriba en el eje Y, lo que haremos será modificar la propiedad Transform/Position/Y del personaje. ¿Cómo hacemos esto desde el código?

A través del componente RigidBody2D del personaje. Recuerda que el RigidBody es un componente que nosotros añadimos al personaje y que se encarga de la física del mismo.

```
// Update is called once per frame
void Update()
{

    if(Input.GetMouseButtonDown(0)){
        Debug.Log("Hola");
        GetComponent<Rigidbody2D>().AddForce(new Vector2(0,fuerzaSalto));
```

Vamos a comprobar, ejecutando el juego que el personaje salta:

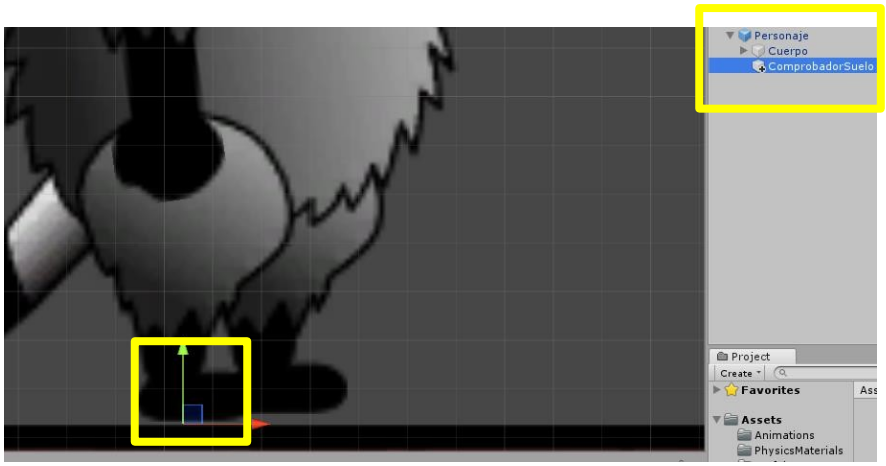
- Si aún no lo tienes, añade un bloque debajo del personaje en la escena y comprueba el funcionamiento del script.

- Observa la propiedad Transform y la pestaña Scene. Dan información de lo que pasa. Es muy útil ver estas opciones cuando se está ejecutando el juego.
- Habrás observado los siguientes problemas:
    - Problema 1, el salto es muy pequeño: Hay que aplicar más fuerza de salto.
    - Problema 2, el personaje parece que flota: Aumentar la fuerza de la gravedad en el rigidbody del personaje. También se podría aumentar para todo el motor de físicas en Edit->Project Settings->Physics2D.
    - Problema 3, se pueden hacer muchos clicks y el personaje sube hasta el infinito: Vamos a limitar el número de saltos consecutivos a 2.

Limitar el número de saltos consecutivos

Solamente permitiremos hacer **dos saltos consecutivos**. El primer salto lo podrá hacer si está tocando el suelo (en realidad, tocando un bloque). Solamente dejaremos hacer un salto, si no está tocando el suelo.

Creamos un **objeto vacío dentro del personaje de nombre ComprobadorSuelo**. Lo ubicamos justo debajo de las patas del personaje.



Ahora desde el código necesitamos saber si toca o no con el suelo para saber si puede dar un segundo salto.

Vamos al script y creamos:

- Una **variable privada booleana llamada enSuelo e inicializada a true.**
- Una **variable pública comprobadorSuelo de tipo Transform** para obtener la posición del objeto del personaje.
- Una **variable privada comprobadorRadio de tipo float con valor 0,07f**, para incluir un radio en el objeto comprobadorSuelo.
- Una **variable pública mascaraSuelo de tipo LayerMask**, para saber lo que es suelo. Todos los bloques van a pertenecer a una capa.

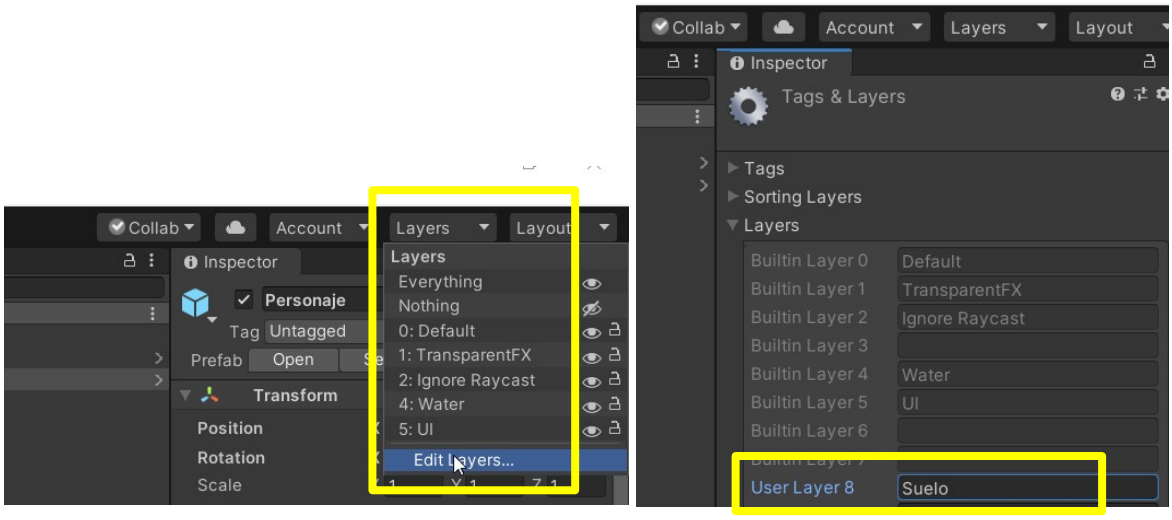
```
//Variables para comprobar si el personaje toca suelo
private bool enSuelo = true;
public Transform comprobadorSuelo;
float comprobadorRadio = 0.07f;
public LayerMask mascaraSuelo;
```

El personaje toca el suelo si comprobadorSuelo con el radio comprobadorRadio colisiona con mascaraSuelo.

Para hacer **comprobaciones en el motor de físicas** utilizamos el **método FixedUpdate().** Esto permite que el código se ejecute cada cierto tiempo físico, independientemente de los fotogramas por segundo que procese el ordenador. Aquí vamos a calcular si el personaje está tocando el suelo o no. Utilizaremos el **método OverlapCircle** para saber si toca suelo o no.

Ahora, debemos asignar los objetos correspondientes a las variables del script:

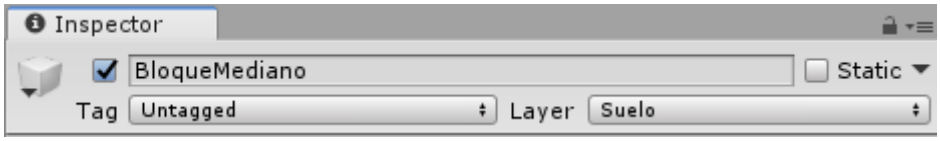
- Creamos una capa suelo:



- Arrastramos el **objeto ComprobadorSuelo a la variable del script.**



- Añadimos la capa a todos los prefabs de bloques.



Para comprobar si funciona correctamente la detección de si toca el suelo el personaje añadiremos un mensaje de log en el método FixedUpdate que muestre en la consola el valor de la variable enSuelo.

```
void FixedUpdate(){

    //comprobamos si comprobadorSuelo+Radio tocan con los objetos que
    //pertenezcan a mascaraSuelo
    enSuelo=Physics2D.OverlapCircle(comprobadorSuelo.position,comprobadorRadio,mascaraSuelo);
     Debug.Log("Toca Suelo"+ enSuelo);
```

Ejecuta el juego, haz algún salto y revisa los mensajes de la consola.

Para que solo detecte un click y salte cuando toca el suelo hay que añadir al método Update lo siguiente:

```
void Update()
{

    if(enSuelo && Input.GetMouseButtonDown(0)){
        GetComponent<Rigidbody2D>().AddForce(new Vector2(0,fuerzaSalto));
```

Ya podemos limitar los saltos a 2. Para ello:

- Crea una **variable booleana dobleSalto**. Que por defecto es false.
- Programa el doble salto en los métodos FixedUpdate y Update:

```
void FixedUpdate(){

    //comprobamos si comprobadorSuelo+Radio tocan con los objetos que
    //pertenezcan a mascaraSuelo
    enSuelo=Physics2D.OverlapCircle(comprobadorSuelo.position,comprobadorRadio,mascaraSuelo);
     Debug.Log("Toca Suelo"+ enSuelo);

     // Si vuelve a tocar el suelo se desactiva el doble salto
     if(enSuelo){
         dobleSalto=false;
     }
}
```

```
// Update is called once per frame
void Update()
{

    if((enSuelo || !dobleSalto) && Input.GetMouseButtonDown(0)){

        GetComponent<Rigidbody2D>().AddForce(new Vector2(0,fuerzaSalto));


        // Comprobar lo del doble salto
        if(!enSuelo && !dobleSalto){
            dobleSalto=true;
        }


    }
```

Tan solo nos queda hacer que se **ejecute la animación Jump** cuando el personaje salta. Para ellos tenemos que modificar el valor de la variable isGrounded, que controla la transición Stand/Jump. Esta variable hay que modificarla en el componente Animator del personaje. Obtendremos este componente de la misma forma que obtuvimos el RigiBody2D.

```
void FixedUpdate(){

    //comprobamos si comprobadorSuelo+Radio tocan con los objetos que
    //pertenezcan a mascaraSuelo
    enSuelo=Physics2D.OverlapCircle(comprobadorSuelo.position,comprobadorRadio,mascaraSuelo);
    Debug.Log("Toca Suelo"+ enSuelo);

    //Se modifica la variable isGrounded del animator
    GetComponent<Animator>().SetBool("isGrounded",enSuelo);

    // Si vuelve a tocar el suelo se desactiva el doble salto
    if(enSuelo){
        dobleSalto=false;
    }
}
```

Ejecuta el juego y comprueba el resultado. Fíjate en la vista del animator, verás cómo se producen las transiciones.

9.- Programación Correr

Para hacer que el personaje corra vamos a realizar las siguientes tareas:

- Hacer que el personaje se mueva en el eje X cuando se pulse con el dedo en la pantalla o se haga clic con el botón izquierdo del ratón.
- Hacer que se ejecute la animación Run.
- Hacer que la cámara siga al personaje cuando este se desplace.

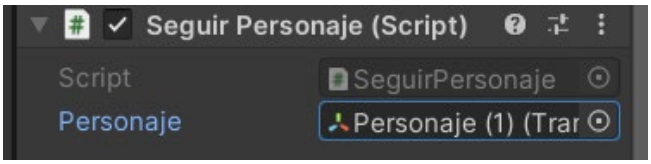
Cámara sigue al personaje

Vamos a crear el script SeguirPersonaje y se lo asignaremos a la cámara. Para ello:

- Obtenemos una referencia al componente Transform del objeto que se sigue(en nuestro caso, el personaje):

```
public class SeguirPersonaje : MonoBehaviour
{
    //Referencia al objeto Transform del personaje
    public Transform personaje;
```

- Asignamos el script a la cámara.
- Asignamos el personaje al script SeguirPersonaje.



- Vamos a hacer en el método Update del script SeguirPersonaje que la cámara modifique su posición, teniendo en cuenta la posición del personaje.

```
// Update is called once per frame
void Update()
{
    transform.position=new Vector3(personaje.position.x, transform.position.y, transform.position.z);
}
```

Ejecuta el juego y mueve, desde el inspector de propiedades, la posición X del personaje para comprobar el desplazamiento.

Una opción para mejorar es que la cámara vaya un poco adelantada respecto al personaje, para ello:

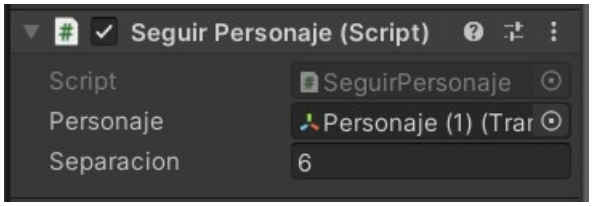
- Define una variable pública float separacion.
- Inicialízala en las propiedades del script de la cámara.
- Añádela al código anterior y ajusta su valor.

```
//Referencia al objeto Transform del personaje
public Transform personaje;
// Separación para que la cámara vaya más adelantada
public float separacion;

// Start is called before the first frame update
void Start()
{

}

// Update is called once per frame
void Update()
{
    transform.position=new Vector3(personaje.position.x+separacion, transform.position.y, transform.position.z);
}
```



Movimiento del personaje (correr)

Vamos a hacer que el personaje empiece a correr. Cuando pulsemos el ratón, se empezará a mover hacia adelante. Sólo puede empezar a correr cuando esté parado. Trabajaremos en el script ControladorPersonaje.

Crearemos una **variable booleana corriendo** para saber su estado y decidir si tiene que empezar a correr o no es necesario porque ya está corriendo.

```
// Update is called once per frame
void Update()
{

    if(Input.GetMouseButtonDown(0)){
      if(corriendo){
        //Hacemos que salte si puede saltar con el código implementado anteriormente
       if(enSuelo || !dobleSalto){

          GetComponent<Rigidbody2D>().AddForce(new Vector2(0,fuerzaSalto));
           //GetComponent<Rigidbody2D>().AddForce(Vector2.up * fuerzaSalto);

         // Comprobar lo del doble salto
         if(!enSuelo && !dobleSalto){
           dobleSalto=true;
       }
      }

    }else{
        corriendo=true;
    }
```

En el **FixedUpdate**, aplicamos una velocidad en el eje X si el personaje estaba parado.

Utiliza una variable pública velocidad para poder ajustar la velocidad a la que corre el personaje. No olvides darle un valor en las propiedades del personaje, en el script.

```
void FixedUpdate(){

    if(corriendo){
       GetComponent<Rigidbody2D>().velocity=new Vector2(velocidad, GetComponent<Rigidbody2D>().velocity.y);
    }

    //comprobamos si comprobadorSuelo+Radio tocan con los objetos que
    //pertenezcan a mascaraSuelo
    enSuelo=Physics2D.OverlapCircle(comprobadorSuelo.position,comprobadorRadio,mascaraSuelo);
     Debug.Log("Toca Suelo"+ enSuelo);

     //Se modifica la variable isGrounded del animator
    GetComponent<Animator>().SetBool("isGrounded",enSuelo);

     // Si vuelve a tocar el suelo se desactiva el doble salto
     if(enSuelo){
         dobleSalto=false;
     }
}
```

Ejecuta el juego y comprueba el funcionamiento.

Animación Run

Para que se ejecute la animación Run, tenemos que modificar la variable velX del animator, ya que es la que controla el paso de Stand a Run.

Lo hacemos de la misma forma que antes:

```
void FixedUpdate(){

    if(corriendo){
       GetComponent<Rigidbody2D>().velocity=new Vector2(velocidad, GetComponent<Rigidbody2D>().velocity.y);
    }

    //Se modifica la variable float del animator
    GetComponent<Animator>().SetFloat("VelX",GetComponent<Rigidbody2D>().velocity.x);
```

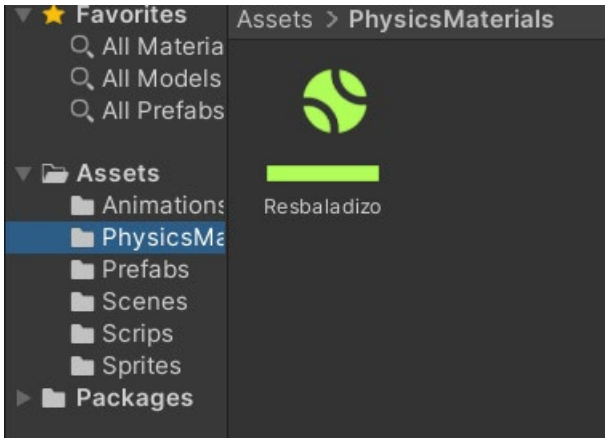
Añade unos cuantos bloques a la escena y comprueba que funciona. Ajusta los parámetros de velocidad y salto del personaje si es necesario.

10.- Bloques

Fricción en los bloques

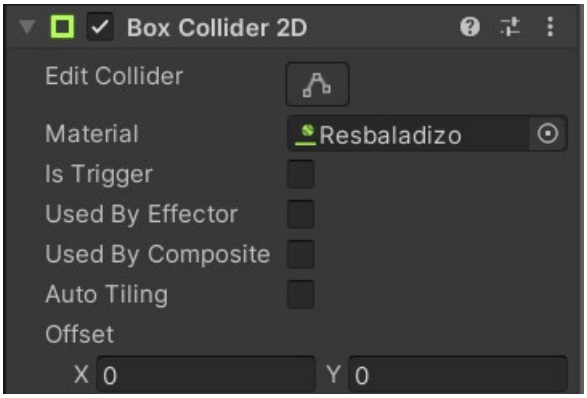
Para evitar que el personaje se atranque con los bloques cuando choque contra ellos de frente, vamos a eliminar la fricción en ellos:

- Creamos una carpeta PhysicsMaterials y dentro de ella un PhysicsMaterial2D de nombre Resbaladizo (Assets/Create/Physics Material 2D).



- Ponemos en sus propiedades la **fricción con valor 0.**

- Ahora añadimos el material creado a los Bloques. Se añade dentro del Collider de cada bloque.



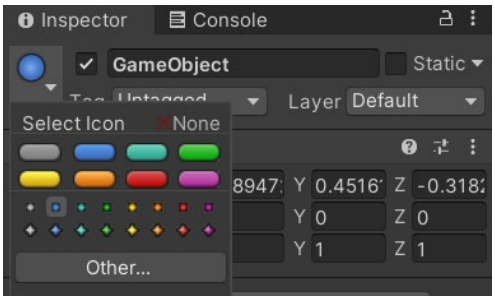
- Si lo haces en los objetos de la escena, recuerda actualizar el prefab de los bloques para que los cambios sean permanentes.

Generar bloques

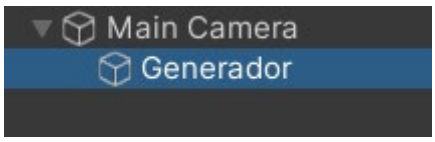
Para que el personaje se vaya moviendo infinitamente, necesitamos que los bloques se vayan generando automáticamente delante del personaje.

Para ello creamos unos elementos generadores cuyo script genera los bloques que queramos en un periodo aleatorio de tiempo que también configuramos:

- Añade un prefab, con el nombre Generador.
- Hacemos un GameObject. Este objeto es el que va a generar los bloques. Y le ponemos por nombre Generador.
- Cambia su icono para que sea visible.



- Arrástralo a la cámara para que dependa de ello y se mueva con ella (delante de ella). Para que sea su hijo.



- Ubícalo a la altura que quieres que genere los bloques. Para ello coge la altura (Y) del bloque largo que tenemos en pantalla y ponemos lo mismo en el objeto Generador.

- Una vez creado el objeto generador, vamos a crear el script que controla su funcionamiento. Este script será el encargado de generar según un intervalo de tiempo, una serie de objetos (en este caso, bloques de distintos tamaños):
    - Crea el **script Generador**.
    - Queremos hacer que, en un intervalo de tiempo aleatorio, genere uno de los tres bloques que tenemos. Para ello el script tiene que recibir esos bloques y dos variables que limitan el margen de tiempo en el que se generará el bloque. Por tanto, se tienes que declarar estas tres variables públicas:

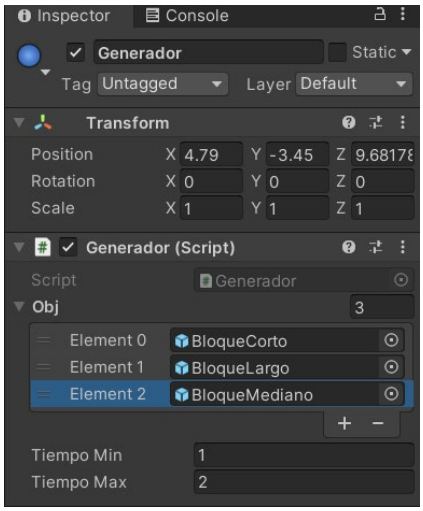
```
//Array con los objetos  que se van a crear de forma
automática
public GameObject[] obj;
//Intervalo de tiempo en el que se van a generar los objetos (mínimo y máximo)
//Se generan objetos de forma aletoria entre 1 y 2
segundos
public float tiempoMin = 1f;
public float tiempoMax = 2f;
```

- En el script creamos un método Generar () que se encarga de elegir uno de entre todos los objetos del array para generarlo.

```
void Generar()
{
    //Genera un objeto en una posición sin rotación
    //Objeto: Uno de los que hay en el array obj, se elige de forma aleatoria
    //Posición: En la posición en la que está en ese momento el generador
    //          (va a ir cambiando ya que se mueve con la cámara)
    Instantiate(obj[Random.Range(0,obj.Length)],transform.position,Quaternion.identity);
}
```

- Llamamos a este método en el Start().

- Asignamos el script al objeto Generador y le indicamos los objetos que va a tener el array que en nuestro caso es bloque largo, bloque corto y bloque medio.



- Probamos que funciona dando play y comprobamos como genera un bloque.

- El método generar se va a llamar a sí mismo, de forma recursiva, cada x segundos. Se genera de forma aleatoria con un valor comprendido entre el tiempo mínimo y máximo configurado.

```
void Start()
{
    Generar();
}

// Update is called once per frame
0 referencias
void Update()
{

}
1 referencia
void Generar()
{
    //Genera un objeto en una posición sin rotación
    //Objeto: Uno de los que hay en el array obj, se elige de forma aleatoria
    //Posición: En la posición en la que está en ese momento el generador
    //          (va a ir cambiando ya que se mueve con la cámara)
    Instantiate(obj[Random.Range(0,obj.Length)],transform.position,Quaternion.identity);
    //Llamada recursiva
    Invoke("Generar", Random.Range(tiempoMin,tiempoMax));
}
```

- Volvemos a comprobar que en la escena van apareciendo bloques de distintos tamaños.
- Añadimos 2 generadores más a la escena para crear los otros 2 niveles. Para ello tenemos que meter esos generadores dentro de la cámara como hijos suyos y además para que no se solapen los bloques vamos a cambiar el tiempo mínimo y máximo a 1,25 y 2,75. También podemos hacer que en el nivel 1 solo se generen bloques grandes y medianos y en los otros niveles los tres tipos de bloques.

11.- Notificaciones

Como has comprobado, nada más iniciar el juego y antes de que el personaje empiece a correr, los bloque se generan y se superponen en la escena.

Para solucionar este problema el personaje va a notificar a esos generadores cuándo empieza a correr.

Para ello utilizaremos una clase generadora y receptora de eventos que está implementada en el script NotificationCenter.cs . Descarga y coloca el script en la carpeta Scripts del proyecto.

Las **notificaciones funcionan de la siguiente forma**:

- **Generar Notificación:** Un objeto, a través de su script, genera una notificación con un nombre concreto. En este caso, el personaje, en el script ControladorPersonaje genera la notificación InicioJuego:

```
NotificationCenter.DefaultCenter().PostNotification(this, "InicioJuego");
```

- **Escuchar Notificación:** Cualquier otro objeto (también a través de su script) puede establecer un listener que escucha una notificación concreta.

```
NotificationCenter.DefaultCenter().AddObserver(this, "InicioJuego");
```

  El objeto que escucha, debe **definir un método** que se ejecutará cuando se genere esa notificación y en el que se programa qué hacer.

Vamos a <u>implementar las notificaciones en el juego</u> para que los generadores empiecen a generar una vez que el personaje esté corriendo:

- Asegúrate de tener el script NotificationCenter en la carpeta de Scripts del proyecto.
- Modifica el script que controla el personaje, para que genere un evento de nombre PersonajeEmpiezaACorrer cuando se produzca esa circunstancia.

```
else
{
    corriendo = true;
    //Lanzamos la notificacición PersonajeEmpiezaACorrer
    NotificationCenter.DefaultCenter().PostNotification(this, "PersonajeEmpiezaACorrer");
}
```

- En el script generador de bloques, añade el listener para la notificación PersonajeEmpiezaACorrer en el método Start().

```
void Start()
{
    NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeEmpiezaACorrer");
}
```

- En el script generador de bloques, crea el método PersonajeEmpiezaACorrer. Debe iniciar la generación de bloques en ese momento.

```
void PersonajeEmpiezaACorrer(Notification notification)
{
    Debug.Log("Notificación del objeto:"+notification.sender.gameObject.name);
    Generar();
}
```

- Comprueba el funcionamiento.

12.- Detección de colisiones

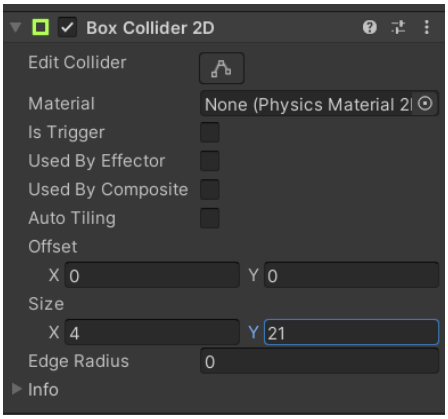
Destrucción de objetos (bloques)

Un problema que nos añaden los generadores de bloques anteriores es que los bloques que generan se mantienen en la escena. Para no sobrecargar el juego innecesariamente, vamos a destruirlos.

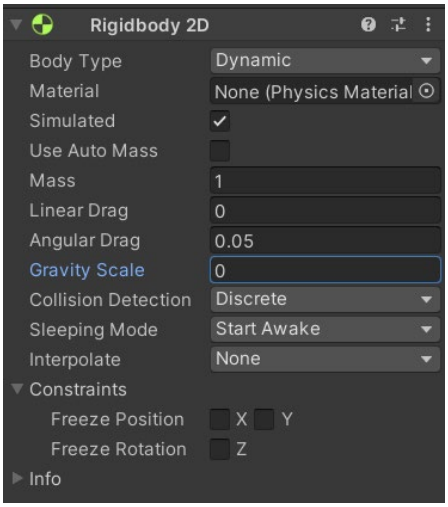


Para ello vamos a crear un objeto situado detrás de la cámara con un Collider que destruirá los bloques cuando colisionen con él:

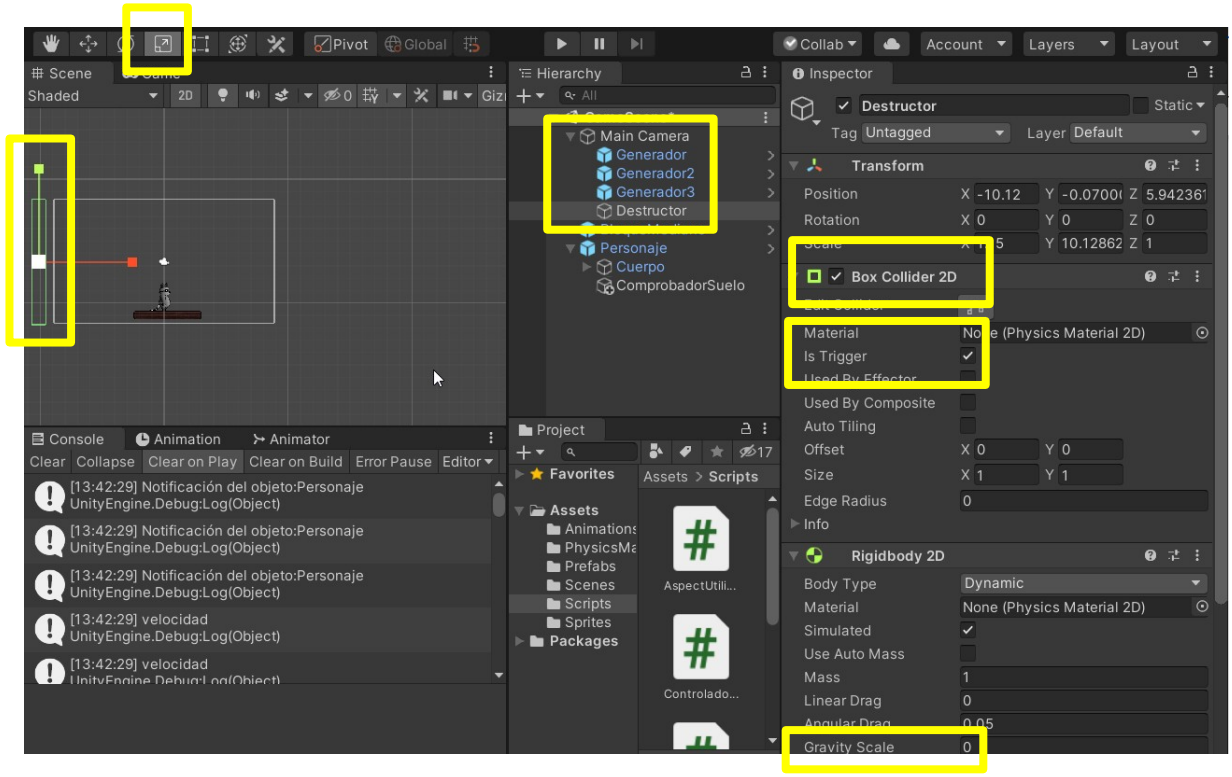
- Crea un objeto vacío. Ponle el nombre de Destructor.
- Añádele un **box collider2d**. Modifica sus dimensiones para que pueda alcanzar a todos los bloques.



- Al Collider le activamos la propiedad IsTriger para que notifique cuando se produzca una colisión.
- Añádele un RigidBody2d. Si no lo hacemos, el Collider no generaría la notificación. Pon “GravityScale”=0 para que no le afecte la gravedad.



- Ubícalo detrás de la cámara en la escena.
- Haz que dependa de la cámara en la jerarquía. Así avanzará a la vez que la cámara.



Vamos a programar que cuando el destructor choque con otro objeto, se destruya dicho objeto:

- Crea el **script Destructor** que implementará el **evento OnTriggerEnter2D** que se lanzará cuando el destructor choque contra otro Collider.
- Cuando haya una colisión, queremos que se destruya el bloque (parámetro collision) con el **método Destroy(objeto).**

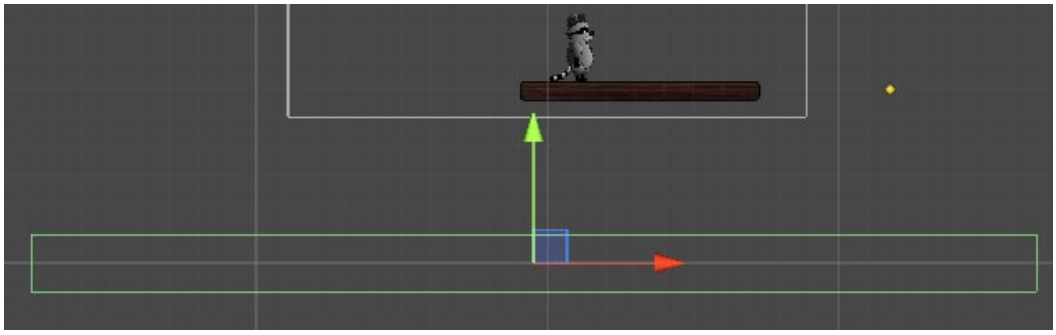
```
private void OnTriggerEnter2D(Collider2D collision)
{
    Destroy(collision.gameObject);
}
```

- Aplica el script al objeto destructor de la escena y comprueba el funcionamiento.

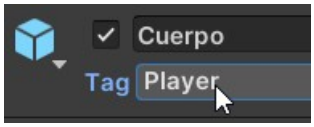
Detección fin de juego

Aprovechando que tenemos este destructor, vamos a reutilizarlo para detectar cuando el personaje se cae, lo cual debería hacer que el juego termine, esta sería la condición de fin de juego:

Copiamos el objeto destructor y lo ponemos debajo de la cámara y comprobamos el funcionamiento.



Cuando el personaje cae se detecta la colisión con uno de los Colliders de las patas. El código va a destruir ese objeto, no el personaje completo. Para solucionarlo vamos a poner el **tag de nombre player** a los objetos del personaje que tienen Colliders: cabeza, cuerpo y las dos patas inferiores.



En el evento **OnTriggerEnter** anterior, establecemos una condición. Si la colisión es con el personaje, se acaba el juego. Más adelante programaremos qué se hace cuando se acaba el juego, habrá que mostrar una pantalla de fin de juego y mostrar la puntuación máxima, etc… Codifícalo en el script.

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.tag == "Player")
    {
        Debug.Log("Fin Juego");
    }
    Destroy(collision.gameObject);
}
```

13.- Ganando puntos

Para hacer más completo el juego, vamos a plantear el sistema de puntos:

- El jugador gana 1 punto cuando el personaje salte encima de una plataforma: Se detectará cuando el jugador colisiona con un bloque.
- Ganará 3 puntos cuando recoja un objeto ítem: Se detectará cuando el jugador colisiona con un ítem.

Lo primero de todo será crear un **script llamado Puntuación**. En este script creamos una **variable entera privada puntuación** inicialmente con valor 0, que irá incrementándose cuando se deban ganar puntos. A través del sistema de eventos, incrementamos esa variable cuando el personaje colisione con un bloque. Por tanto, tenemos que establecer un listener que haga esa tarea.

```
public class Puntuacion : MonoBehaviour
{
    //Variable que almacena la puntuación
    private int puntuacion = 0;
    // Start is called before the first frame update
    0 referencias
    void Start()
    {
        //Añadimos un listener a la notificación IncrementarPuntos
        NotificationCenter.DefaultCenter().AddObserver(this, "IncrementarPuntos");
    }
    //Increpentamos puntuación
    0 referencias
    void IncrementarPuntos(Notification notification)
    {

    }
```

Los puntos que hay que incrementar los vamos a recoger del objeto notificación; si la notificación la lanza un bloque será 1 punto y si la lanza un ítem, serán 3. Por el momento, mostraremos la puntuación en un mensaje de log.

```
void IncrementarPuntos(Notification notification)
{
    //Recogemos los puntos a incrementar de la notificación
    int puntosAIncrementar = (int) notification.data;
    //Modificamos la puntación
    puntuacion += puntosAIncrementar;
    //Mostramos por consola la puntación
    Debug.Log("Puntuación:" + puntuacion);
}
```

El **script Puntuación** debemos asignárselo a un objeto independiente y permanente en la escena, por ejemplo, la cámara. **Asignamos el script a la cámara**.

Los objetos que generan la notificación para incrementar puntos son los bloques y lo items. Crearemos un nuevo **script llamado IncrementarPuntos** y se lo asignaremos a estos objetos. Cuando se produzca una colisión de los objetos con el personaje, se lanzará la notificación IncrementarPuntos con los puntos a incrementar. Este script tendrá una variable entera pública que almacenará los puntos que incrementa cada objeto (1 los bloques y 3 los items):

Como lo dos objetos que colisionan (Personaje y bloques o items) tienen Colliders, programamos el método OnCollisionEnter2D (en la colisión con el destructor capturamos el evento OnTriggerEnter2D ya que estaba activo en el Collider la propiedad isTrigger) y lanzamos la notificación.

```
public class IncrementarPuntos : MonoBehaviour
{
    //Puntos a incrementar
    public int puntos;
    // Start is called before the first frame update
    0 referencias
    void Start()
    {

    }

    // Update is called once per frame
    0 referencias
    void Update()
    {

    }
    0 referencias
    private void OnCollisionEnter2D(Collision2D collision)
    {
        NotificationCenter.DefaultCenter().PostNotification(this, "IncrementarPuntos", puntos);
    }
```

Asigna el script IncrementarPuntos al prefab de cada tipo de bloque y establece el valor de puntos en 1. Comprueba el funcionamiento.

Al comprobar el funcionamiento se detectan dos problemas:

- La puntuación se incrementa varias veces en cada bloque: debemos crear una variable booleana que controle una sola colisión por bloque.
- La puntuación se incrementa cuando el bloque colisiona con el destructor que va detrás de la cámara: solamente tenemos que incrementar la puntuación si la colisión se ha hecho con el personaje. Asigna al personaje la etiqueta Player para poder hacer esta comprobación.

```
public class IncrementarPuntos : MonoBehaviour
{
    //Puntos a incrementar
    public int puntos;
    //Comprobar si ya se ha incrementado
    bool haColisionado = false;

    // Start is called before the first frame update
    0 referencias
    void Start()
    {

    }

    // Update is called once per frame
    0 referencias
    void Update()
    {

    }
    0 referencias
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (!haColisionado && collision.gameObject.tag == "Player") {
            NotificationCenter.DefaultCenter().PostNotification(this, "IncrementarPuntos", puntos);
            haColisionado = true;
        }
```

Es el momento de **añadir items a la escena**. Añade un ítem a la escena, asígnale el script IncrementarPuntos y establece la puntuación a 3 (sobrescribe los cambios en el prefab). Comprueba el funcionamiento. Verás que al chocar con el ítem, el personaje queda atascado, para solucionar esto, debes **marcar IsTrigger en el Collider del ítem** (sobrescribe el prefab).

Ahora verás que el ítem no incrementa la puntuación y no desaparece cuando el personaje colisiona con él. Esto se debe porque al activar IsTrigger ya no se captura el evento OnCollisionEnter2D sino que se **activa OnTriggerEnter2D**. Modifica el script para capturar el evento OnTriggerEnter2D e incrementar la puntuación además de destruir el objeto ítem en la colisión.

```
//Colisión con un bloque
0 referencias
private void OnCollisionEnter2D(Collision2D collision)
{
    if (!haColisionado && collision.gameObject.tag == "Player") {
        NotificationCenter.DefaultCenter().PostNotification(this, "IncrementarPuntos", puntos);
        haColisionado = true;
    }
}
//Colisión con un Item
0 referencias
private void OnTriggerEnter2D(Collider2D collision)
{

    if (!haColisionado && collision.gameObject.tag == "Player")
    {
        NotificationCenter.DefaultCenter().PostNotification(this, "IncrementarPuntos", puntos);
        haColisionado = true;
        //Destruímos el item
        Destroy(this.gameObject);
    }
}
```

Solamente nos queda que los **items se generen de forma aleatoria** y para hacerlo reutilizaremos el script de generador de bloques:

- Duplica un generador de bloques. Cámbiale el nombre y el icono.
- Haz que genere items en lugar de bloques quitando del array los bloques y añadiendo al array el ítem.
- Posiciónalo en la escena de forma correcta.
- Ajusta el tiempo de aparición en la escena.
- Puedes añadir varios generadores de items a la escena.



14.- Marcador de puntos

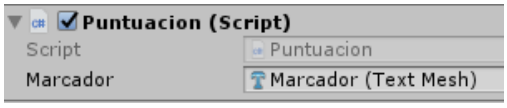
Vamos a colocar el **marcador de puntos en la esquina superior derecha**. Mostrará los puntos conseguidos que se almacenan en la variable puntuación del script Puntuación:

- Añadimos un objeto de texto:
  - GameObject/3D Object/Legacy/Text Mesh.
  - Le ponemos el nombre Marcador.
  - Lo ponemos dentro de la cámara.
  - Lo colocamos en la parte superior derecha.
  - Cambiamos la **propiedad TextMesh/Text** para que por defecto aparezca un número de varias cifras, por ejemplo, 1234.
  - Cámbiale el punto de anclaje. **Propiedad Anchor con el valor Upper Left**.
- Importar unas fuentes al proyecto.
  - Creamos la carpeta Fonts y colocamos en ella la fuente que está disponible en la carpeta moodle de la unidad 5.
- Cargar las fuentes y configurar el tamaño del texto.
  - Aplica la fuente a la propiedad Font.
  - Edita la fuente (doble clic sobre la fuente).Cuanto mayor sea el tamaño de la fuente, mejor será su textura. Pon un tamaño de 32 px.
  - Disminuir el tamaño del objeto de texto.
- Si los cambios no se aplican en la **vista de juego** hay que modificar la rotación Z.

- **Actualizar el valor del marcador con la puntuación real:**
  - Hay que hacer que ese script Puntuación que está asignado a la cámara, tenga acceso al marcador para actualizar el valor de la propiedad TextMesh/Text. Lo haremos **creando una variable pública de tipo TextMesh** en el script puntación.

```
public class Puntuacion : MonoBehaviour
{
    //Variable que almacena la puntuación
    private int puntuacion = 0;
    //Variable para enlazar con el marcador
    public TextMesh marcador;
```

- Arrastramos el objeto marcador para enlazar su componente TextMesh con la variable pública del script.



- Ahora solo queda actualizar el marcador cuando se modifique la puntuación.

```
//Actualizamos el marcador
marcador.text = puntuacion.ToString();
```

- Inicializa el marcador a 0 y prueba el resultado.

15.- Fondos/Scroll Parallax

En esta presentación veremos cómo añadir fondos a la escena. Añadiremos animaciones y profundidad, lo que se conoce como **efecto parallax** dónde, el fondo cercano se mueve muy rápido y el fondo lejano, más lento.

Añadimos y colocamos los fondos:

- Crea la carpeta Texturas e importa los fondos que vamos a utilizar que están en la plataforma moodle llamado texturas. Tienen que tener el valor Default en la propiedad Texture Type.
- El **FondoCerca** se va a situar delante del otro fondo. Hay que hacer que sea transparente.

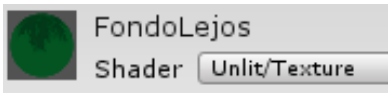


- También hacemos que los fondos sean cíclicos, se repitan.



- Para **añadir el fondo a la escena**, creamos un **objeto Quad** (Game Objext/3D Objext/Quad) de nombre FondoLejos hijo de la cámara y con el mismo tamaño que la cámara.
- Le aplicamos la textura FondoLejos (arrastra el fondo al objeto quad).

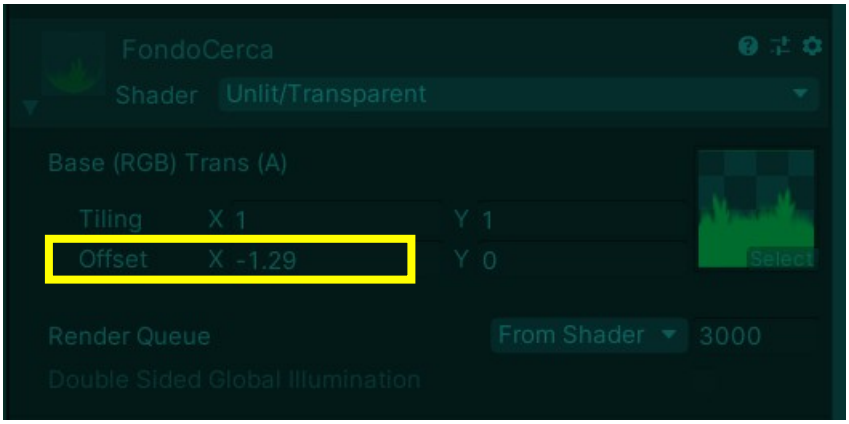
- El fondo no se ve en la escena de juego. Hay que moverlo en el eje Z.
- Ahora el fondo se ve oscuro. La razón es que a su Shader, le afecta la luz. Hay que cambiarlo:



- Hacemos lo mismo para **FondoCerca**:
  - Duplicamos el anterior y le cambiamos la textura.
  - Posiciónalo, en el eje Z, ligeramente más cerca de la cámara que FondoLejos.
  - En este caso, el Shader va a ser de tipo Unlit/Transparent.

Ejecuta el juego y revisa que todos los componentes se visualizan y no quedan detrás de los fondos.

Para **conseguir el efecto Parallax** vamos a jugar con la propiedad Offset en el eje X de los fondos. Observa cómo se comportan los fondos cuando modificamos el valor del offset de X. Eso es lo que tenemos que hacer desde código para simular el scroll.

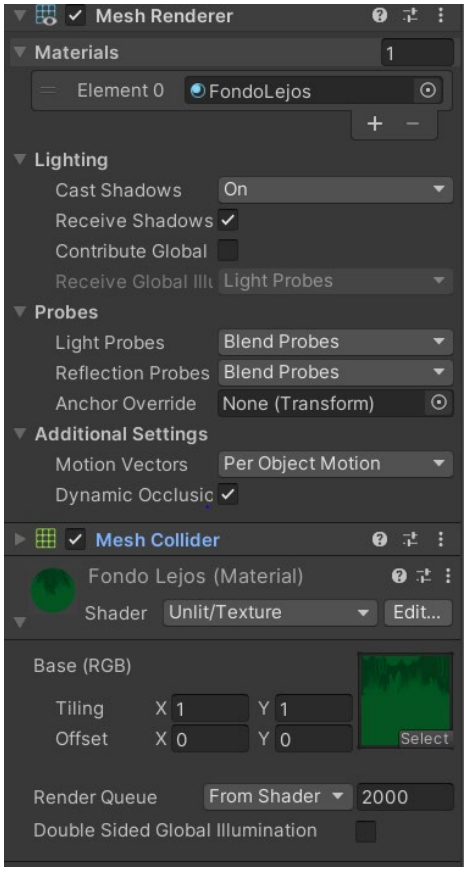


Creamos un script nuevo llamado Scroll:

- Este script se le va a aplicar a cada uno de los dos objetos fondo de la escena.
- Los fondos se van a desplazar a una velocidad diferente, así que definimos una variable pública real velocidad.

```
public float velocidad = 0f;
```

- En el método Update debemos acceder al offset del material. Para ello tenemos que obtener una referencia al Renderer lo que no permitiría acceder a su Material (que es FondoCerca/FondoLejos) y del material a su propiedad Offset.




```
GetComponent<Renderer>().material.mainTextureOffset
```

- Para actualizar el valor del Offset de X, multiplicamos la velocidad por una variable dependiente del tiempo. Usamos la clase time para ello. El Offset de Y se mantiene a 0, su valor original:

```
GetComponent<Renderer>().material.mainTextureOffset=new Vector2(Time.time * velocidad, 0);
```

```
0 referencias
public class Scroll : MonoBehaviour
{
    //Velocidad
    public float velocidad = 0f;
    // Start is called before the first frame update
    0 referencias
    void Start()
    {

    }

    // Update is called once per frame
    0 referencias
    void Update()
    {
        GetComponent<Renderer>().material.mainTextureOffset = new Vector2(Time.time * velocidad, 0);
    }
}
```

- Ahora agrega el script a los fondos.
- Fija una velocidad para cada fondo. Más rápida para el fondo de cerca (0,3) y más lenta para el fondo de lejos (0,1).
- Ejecuta el juego y comprueba el funcionamiento.

- Como habrás observado, al inicio del juego, el personaje está parado pero el fondo se mueve. Para solucionar este problema, podemos capturar la notificación PersonajeEmpiezaACorrer e iniciar el efecto en ese momento.

```
public class Scroll : MonoBehaviour
{
    //Velocidad
    public float velocidad = 0f;
    //Controla si el juego se ha iniciado
    bool enMovimiento = false;
    // Start is called before the first frame update
    0 referencias
    void Start()
    {
        NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeEmpiezaACorrer");
    }
    0 referencias
    void PersonajeEmpiezaACorrer(Notification notification)
    {
        enMovimiento = true;
    }
    // Update is called once per frame
    0 referencias
    void Update()
    {
        if (enMovimiento)
        {
            GetComponent<Renderer>().material.mainTextureOffset = new Vector2(Time.time * velocidad, 0);
        }
    }
}
```

- Ahora se nos presenta un nuevo problema y es que si tardamos unos segundos en iniciar a correr, se produce un salto en el fondo. Es debido a que hemos usado la clase Time para desplazar los fondos. Tenemos que restar el tiempo que hemos tardado en poner en movimiento al personaje.

```
public float velocidad = 0f;
//Controla si el juego se ha iniciado
bool enMovimiento = false;
float tiempoInicio;
// Start is called before the first frame update
0 referencias
void Start()
{
    NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeEmpiezaACorrer");
}
0 referencias
void PersonajeEmpiezaACorrer(Notification notification)
{
    enMovimiento = true;
    tiempoInicio = Time.time;
}
// Update is called once per frame
0 referencias
void Update()
{
    if (enMovimiento)
    {
        GetComponent<Renderer>().material.mainTextureOffset = new Vector2(((Time.time - tiempoInicio) * velocidad)%1, 0);
    }
}
```

16.- Fin del juego

En este apartado vamos a programar las acciones que se deben llevar a cabo cuando finaliza el juego. En ese momento, se detendrá el scroll, se pararán los generadores, se desactivará el personaje y se mostrará una pantalla de fin de juego.

La condición de fin de juego es que el personaje se caiga y choque contra el destructor inferior. En el **script Destructor tenemos un trigger** que cuando detecta una colisión con el personaje detiene el juego. Vamos a trabajar ahí para generar una notificación de fin de juego.

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.tag == "Player")
    {
        NotificationCenter.DefaultCenter().PostNotification(this, "PersonajeHaMuerto");
    }
    Destroy(collision.gameObject);
}
```

Esta notificación deben escucharla:

- **El scroll**: La animación de los fondos se debe detener.

```
public class Scroll : MonoBehaviour
{
    //Velocidad
    public float velocidad = 0f;
    //Controla si el juego se ha iniciado
    bool enMovimiento = false;
    float tiempoInicio;
    // Start is called before the first frame update
    0 referencias
    void Start()
    {
        NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeEmpiezaACorrer");
        NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeHaMuerto");
    }
    0 referencias
    void PersonajeEmpiezaACorrer(Notification notification)
    {
        enMovimiento = true;
        tiempoInicio = Time.time;
    }

    0 referencias
    void PersonajeHaMuerto(Notification notification)
    {
        enMovimiento = false;
    }
```

- **Los generadores de bloques e items:** No deben generar más objetos:

```
public class Generador : MonoBehaviour
{
    //Array con los objetos  que se van a crear de forma automática
    public GameObject[] obj;
    //Intervalo de tiempo en el que se van a generar los objetos (mínimo y máximo)
    //Se generan objetos de forma aletoria entre 1 y 2 segundos
    public float tiempoMin = 1f;
    public float tiempoMax = 2f;

    bool finJuego = false;

    // Start is called before the first frame update
    0 referencias
    void Start()
    {
        NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeEmpiezaACorrer");
        NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeHaMuerto");
    }
    0 referencias
    void PersonajeHaMuerto(Notification notification)
    {
        finJuego = true;
    }

  void Generar()
  {
      //Genera un objeto en una posición sin rotación
      //Objeto: Uno de los que hay en el array obj, se elige de forma aleatoria
      //Posición: En la posición en la que está en ese momento el generador
      //          (va a ir cambiando ya que se mueve con la cámara)
      Instantiate(obj[Random.Range(0,obj.Length)],transform.position,Quaternion.identity);
      //Llamada recursiva
      if (!finJuego) {
          Invoke("Generar", Random.Range(tiempoMin, tiempoMax));
      }

  }
```

- **Desactivar el personaje** para evitar que siga cayendo:

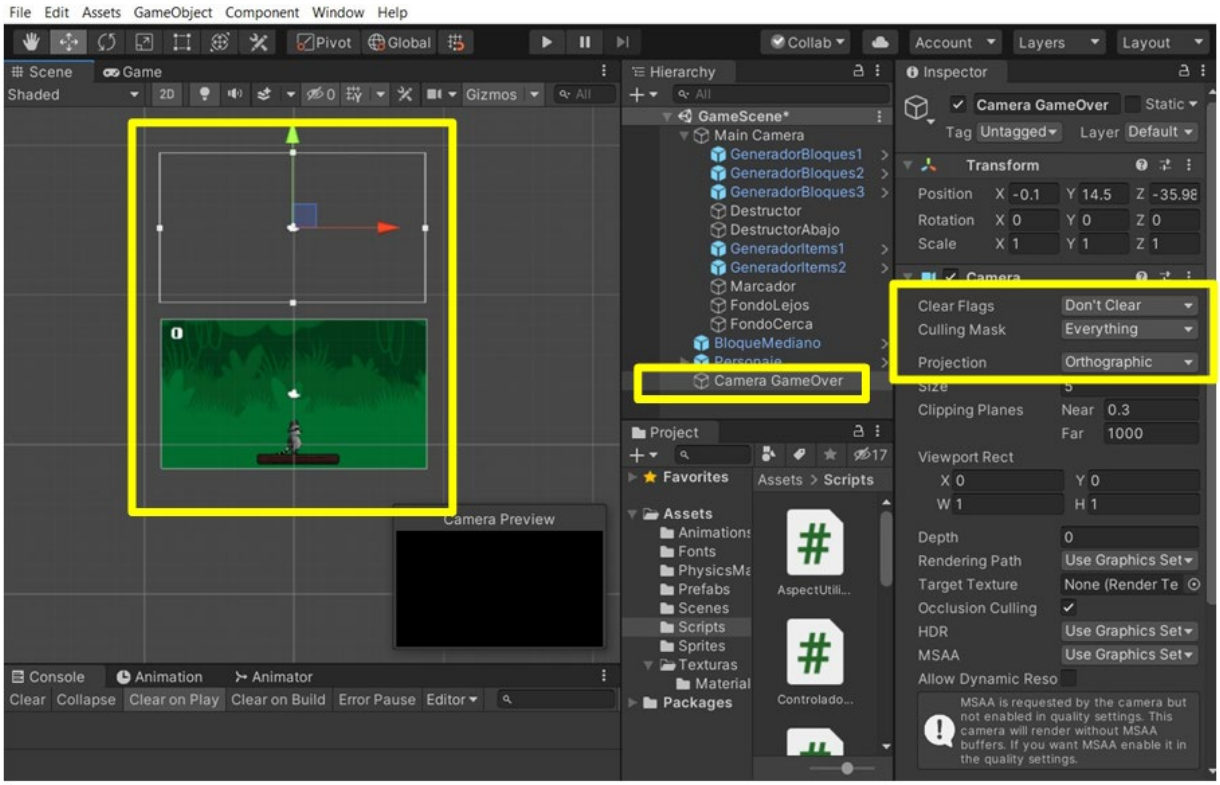
```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.tag == "Player")
    {
        NotificationCenter.DefaultCenter().PostNotification(this, "PersonajeHaMuerto");
        GameObject personaje = GameObject.Find("Personaje");
        personaje.SetActive(false);
    }
    Destroy(collision.gameObject);
}
```

- **Mostrar una pantalla de fin de juego**: Esta pantalla mostrará un mensaje de Game Over, la puntuación obtenida y un botón para volver a jugar. Para ello:

    - El **efecto de la pantalla de Game Over** lo conseguiremos oscureciendo la pantalla actual de juego y creando una cámara con la información de game over. Esta cámara se activa cuando se acaba el juego (y no antes) y se renderiza delante de la cámara de juego.

- Creamos una cámara llamada Camara GameOver (Game Object/Camera). Haz que sea el atributo Projection sea de tipo Orthographic.
- Le cambiamos la propiedad: 

- Le aplicamos el script “AspectUtility” con el valor 1,7777777 para que su relación de aspecto sea 16:9 (como la cámara principal).
- La ubicamos “encima” de la cámara principal en la escena.



- Para que la **pantalla se oscurezca** cuando se activa la cámara de GameOver, creamos un **objeto Quad** que dependa de la cámara GameOver y lo hacemos lo suficientemente grande como para que ocupe toda la cámara. Lo echamos hacia atrás en el eje Z para que lo pueda enfocar la cámara.
- Para oscurecer, en la **carpeta Texturas/Materials**, creamos un material GameOverNegro (Create/Material).
- Le ponemos el color negro y la propiedad : 

- Asignamos el material creado al Quad de la cámara de GameOver.
- Ahora jugando con el valor Alpha del color del material, podemos oscurecer la pantalla.

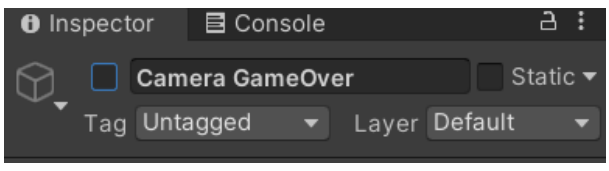


- **Añadimos los textos a la cámara GameOver**. El proceso es el mismo que con el marcador:
    - Objeto 3D text con el texto “Game Over”. Si el texto no aparece en la cámara, posiciónalo en el eje Z detrás de ella (pero delante del Quad).
    - Duplicamos el texto anterior para poner el total de puntos con el texto “Total:”.
    - Y añadimos otro objeto más para poner la puntuación a la derecha de “Total:”.
    - Por último, ponemos un texto para permitir la opción de volver a jugar.





- La cámara Game Over debe estar desactivada. Lo que se pretende hacer desde código es activarla cuando el personaje cae y se detecta el fin de juego. **Desactivar la cámara GameOver:**



- Vamos a hacer el script que cuando detecte el final del juego, active la cámara de GameOver. ¿Dónde lo colocamos?
    - En la cámara de game over, NO. Recuerda, que está deshabilitada y no se ejecutarán sus scripts.
    - En la cámara principal. Puede ser una buena solución. Es el objeto más genérico del juego que tenemos y sigue todo el desarrollo del mismo.
- **Crea el script GameOver**, asígnalo a la cámara principal y programa su funcionamiento:
    - Suscribirse al evento que notifica el fin del juego
    - Activar la cámara de game over.

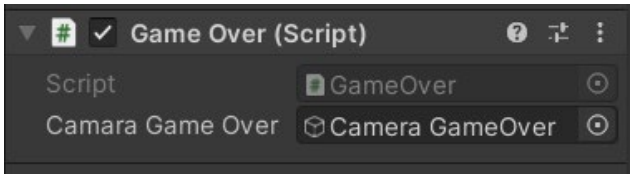
```
public class GameOver : MonoBehaviour
{
    // Objeto de la camara
    public GameObject camaraGameOver;

    // Start is called before the first frame update
    void Start()
    {
        // Notificación al evento cuando personaje cae
        NotificationCenter.DefaultCenter().AddObserver(this, "PersonajeHaMuerto");
    }


    void PersonajeHaMuerto(Notification notification){
        camaraGameOver.SetActive(true);
    }
    // Update is called once per frame
    void Update()
    {

    }
}
```

    - Asignar el script a la cámara principal.
    - Asignar en el script la cámara game over como se muestra en la imagen. Después comprueba su funcionamiento.



- Para poner la **puntuación en el objeto de texto** de la cámara de fin de juego, hay que crear un **script ValoresGameOver** y asígnalo a la cámara game over. Se tiene que encargar de obtener una referencia al objeto de texto donde cargamos el total:

```
public class ValoresGameOver : MonoBehaviour
{

    // refencia al objeto de texto para poner el total puntuación
    public TextMesh textoTotal;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```

- Se asigna el script a la cámara Game Over y se le da el valor al parámetro texto Total.



- **Obtener la puntuación total:** Necesitamos acceder al valor del atributo puntuacion del script Puntuacion.cs. Al ser un atributo privado, añadimos un getter en el script Puntuacion.cs para poder obtener su valor (método GetPuntuacion).

```
public class Puntuacion : MonoBehaviour
{

    //variable que almacena la puntuación
    private int puntuacion=0;

    //variable para enlazar con el marcador
    public TextMesh marcador;

    // Método GetPuntuacion
    public int GetPuntuacion(){
        return puntuacion;
    }
```

  - Además en el script ValoresGameOver hay que definir una variable pública de objeto Puntuacion y asignar P a Main Camera como indica la imagen:

```
// referencia al objeto Puntuacion
public Puntuacion p;
```



- **Cargar el total en el texto**: Lo haremos en el **evento onEnable ()** se lanza cuando se activa el objeto.

```
public class ValoresGameOver : MonoBehaviour
{

    // refencia al objeto de texto para poner el total puntuación
    public TextMesh textoTotal;

    // referencia al objeto Puntuacion
    public Puntuacion p;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }

    private void OnEnable(){
        total.text=p.GetPuntuacion().ToString();
    }
}
```

- **Convertir el texto Jugar en un botón**. Para ello le ponemos un BoxCollider2D al texto.
- Por último, se crea y asigna un **script BotonJugar** en el que capturamos el **evento OnMouseDown()**. Debe cargar la escena del juego.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class BotonJugar : MonoBehaviour
{
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }

    private void OnMouseDown(){
        SceneManager.LoadScene("GameScene");
    }
}
```

17.- Pantalla de portada

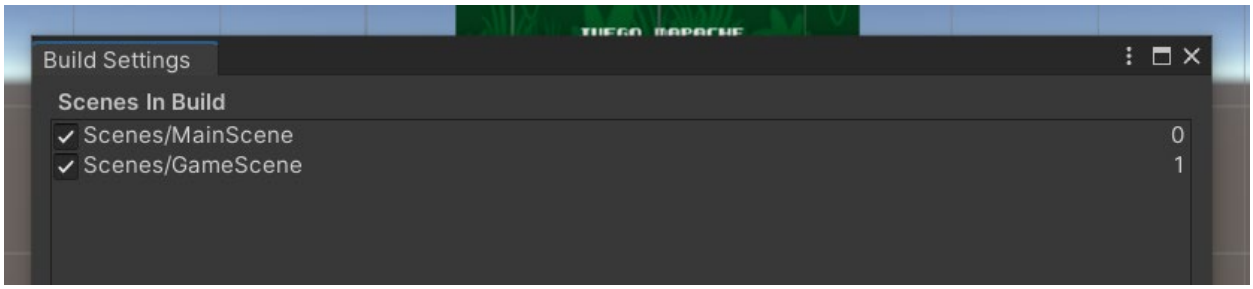
Vamos a crear una pantalla de portada. Va a ser una nueva escena. Tendrá el mismo fondo que la pantalla de juego, texto con información de la aplicación y un botón para volver atrás.

Duplica la escena de juego y ponle el nombre ***MainScene***. Ábrela y elimina de ella todos los objetos excepto los fondos y el marcador.

Ejecuta el juego y observa los errores. La cámara tiene scripts con dependencias sin asignar, corrige los errores (borrar todos los scripts asignados a la cámara).

Reutilizamos el objeto marcador para poner el título del juego, cambia el nombre y la posición del objeto. Debajo del título añade el texto ***Jugar***, para ello duplica el botón jugar que tienes en gameover.

A continuación, tienes que ir a **File/Build Settings** y allí arrastrar las escenas del juego. La que tenga el número más bajo es la escena inicial.



El juego debe tener la siguiente pantalla de portada:



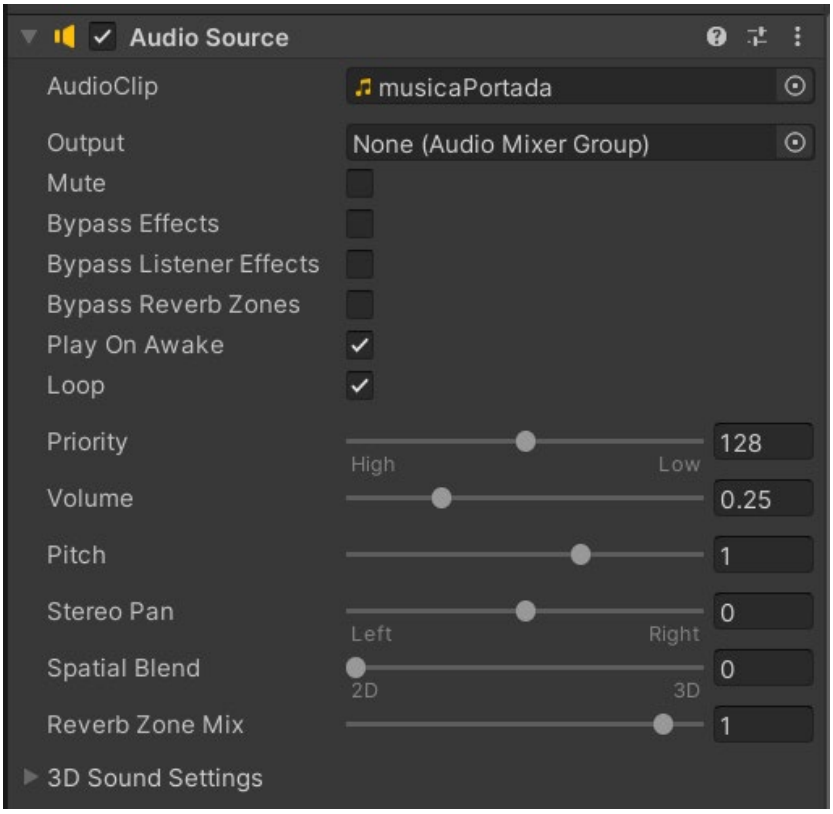
18.- Música y sonidos

Vamos a añadir música y efectos de sonido al juego. Para ello crea la carpeta Sonidos en la carpeta Assets e importa los sonidos proporcionados en la unidad 5 de la plataforma moodle.

Los clips de música que se reproducen constantemente en la escena tendrán que ir asociados a la cámara.

Para ello hay que añadir el **componente Audio Source**:

- Abre la escena de inicio del juego.
- Añade el componente Audio Source a la cámara.
- Añade el clip musicaPortada al componente Audio Source.
- Ajusta los parámetros para que se reproduzca al inicio (Play on Awake) y en bucle (Loop).
- Baja el sonido.
- Prueba el funcionamiento



Para **añadir sonidos a objetos** seguimos el mismo proceso, pero el componente de sonido se añade al objeto en cuestión y no se reproduce al inicio ni en bucle. La reproducción está condicionada a la ocurrencia de un evento.

Vamos a **añadir sonido al botón Jugar**:

- Añade el componente AudioSource al botón Jugar con el sonido pulsarBoton.
- Haz que no se reproduzca al inicio.
- Baja el volumen del sonido.
- Abre el script del botón Jugar.
- Accede a su componente "AudioSource" y en el evento de clic inicia su reproducción.

```
public class BotonJugar : MonoBehaviour
{
    AuidoSource sonido;
    // Start is called before the first frame update
    void Start()
    {
        sonido= GetComponent<AuidoSource>();
    }

    // Update is called once per frame
    void Update()
    {

    }

    private void OnMouseDown(){
        SceneManager.LoadScene("GameScene");
        sonido.Play();

    }
}
```

Hay un problema y es que al pulsar se reproduce el sonido, pero se corta porque se pasa inmediatamente a otra escena. Es posible solucionarlo añadiendo un pequeño retardo cuya duración sea la duración del sonido (usamos el método **Invoke** que ya vimos en el script Generador).

```
private void OnMouseDown()
{
    //Application.LoadLevel("GameScene");
    sonido.Play();
    Invoke("CargarEscena",sonido.clip.length);

}
private void CargarEscena()
{
    SceneManager.LoadScene("GameScene");
}
```

Vamos a añadir ahora el clip de **audio musicaJuego a la escena GameScene**, igual que hemos hecho con la primera.

Realiza las acciones necesarias para **añadir el sonido saltar al personaje** cuando se realice un salto.

Realiza las acciones necesarias para que **suene el sonido item cuando el personaje coge un item.**

Comprueba que hay un problema con el item: el audio no se reproduce. Es debido a que el objeto se destruye inmediatamente a la colisión. Debemos independizar la reproducción del sonido del objeto en sí.

```
if (collision.gameObject.tag == "Player")
{
    //Reproducimos el clip del objeto AudioSource
    //En la posición de la cámara
    //Con un sonido de 0.5f
    AudioSource.PlayClipAtPoint(sonido.clip, Camera.main.transform.position, 0.5f);
    NotificationCenter.DefaultCenter().PostNotification(this, "IncrementarPuntos", puntosGanados);

}
```

Cuando finaliza el juego deberíamos:

- **Parar la reproducción del clip musicaJuego:** el componente que lo reproduce es la cámara. La cámara también tiene un script GameOver que activa la cámara de game over, este es el script ideal para parar la música.

```
void PersonajeHaMuerto(Notification notification)
{
    sonido.Stop();
    camaraGameOver.SetActive(true);
}
```

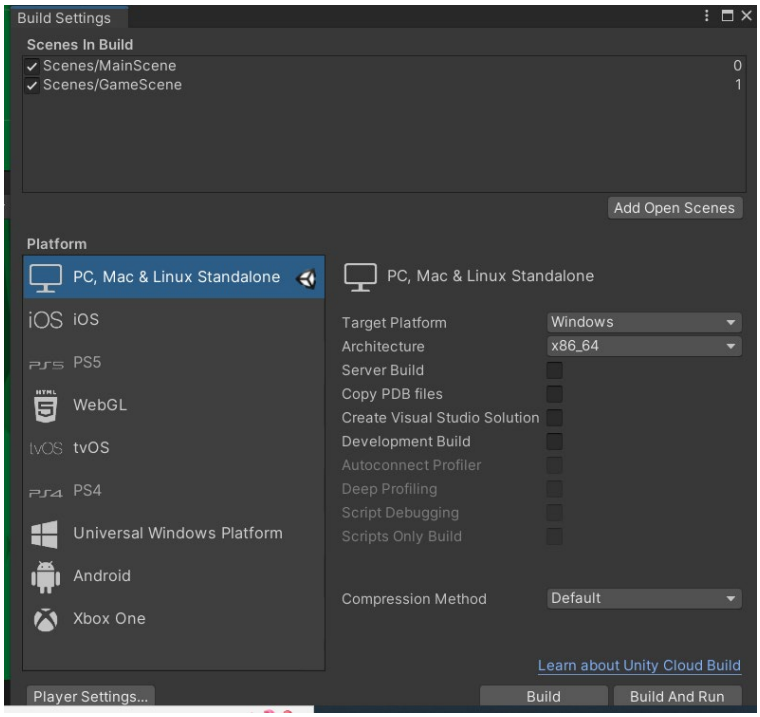
- **Reproducir el clip gameOver:** La reproducción de la música de fin de juego lógicamente se realizará en la cámara de game over. Se reproducirá cuando se active, por tanto, es una reproducción por defecto en la que no hay bucle. No es necesario código:



19.- Exportar el proyecto

Una vez que el juego se dé por terminado tenemos que generar los ejecutables.

Lo primero de todo será decidir qué plataforma queremos exportar, recuerda que Unity es multiplataforma. Si nos vamos a **File->Build Settings** podrás ver las posibilidades que ofrece Unity:

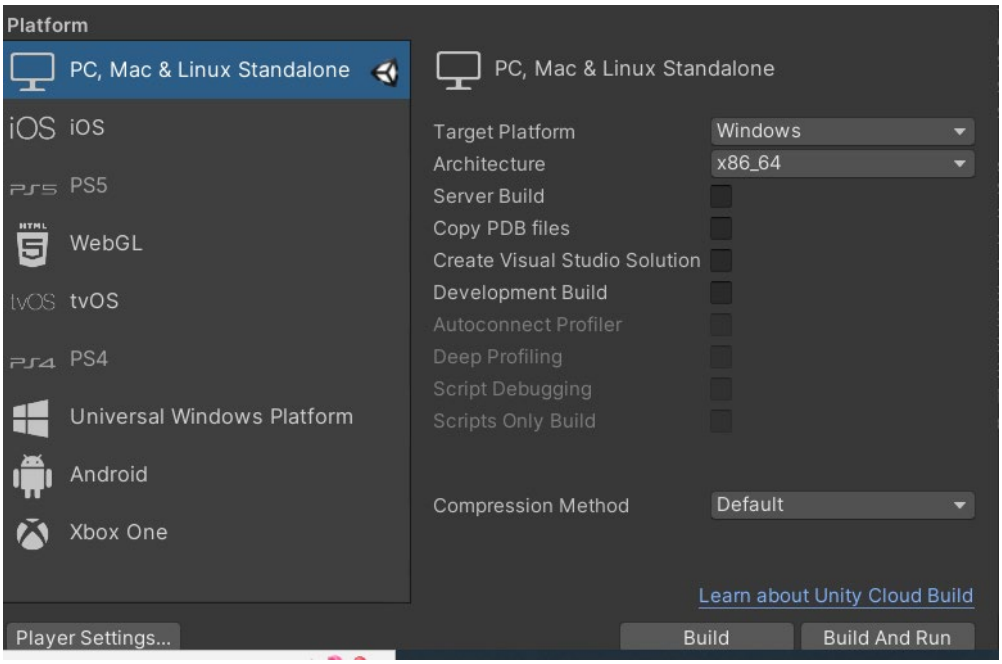


La exportación no es un proceso complejo, aunque hay una serie de requisitos de obligado cumplimiento.

Por ejemplo, para generar un ejecutable Android tenemos que:

- Instalar el plug-in de Unity para Android
- Tener el SDK de android correctamente configurado
- JDK

Haremos una exportación sencilla a PC. Dejamos las opciones que vienen por defecto y pulsamos en "Build" (o "Build and Run" si queremos ejecutarlo a continuación).



Elegimos la carpeta de destino y ya tenemos los ficheros generados:

